💻 Installation
👶 First Steps!
🙉 Headless Setup
🌼 Google Colab
🐳 Docker
🎯 Requirements

💻 Installation

iTHOR comes packaged with AI2-THOR, which can be installed in Python using pip:

```
pip install ai2thor
```

or conda:

```
conda install -c conda-forge ai2thor
```

Optionally, one may also choose to install AI2-THOR from an exact Git commit. Here, we can pass in the commit id to:

```
pip install --extra-index-url https://ai2thor-pypi.allenai.org ai2thor==0+COMMIT_ID
```

For example, we may have:

```
# Example commit id: d74c4a9bf9ec558f27d4bd1779d5e33983b99db0
pip install --extra-index-url https://ai2thor-pypi.allenai.org ai2thor==0+d74c4a9bf9ec558f27d4bd1779d5e33983b99db0
```

👶 First Steps!

To verify that your setup is correct, try initializing a `Controller` and executing an action:

```
from ai2thor.controller import Controller
controller = Controller()
event = controller.step("MoveAhead")
```

The first time a `Controller` is initialized, the game environment containing the 3D scenes will be locally downloaded to `~/.ai2thor`. The size of the download is approximately 500MB.

🙉 Headless Setup

By default AI2-THOR currently opens up a Unity window each time a `Controller` is initialized. However, AI applications often do not need a window that renders the on-screen view and administrators of compute clusters might restrict the use of Xorg on the machines due to security concerns and the large number of dependencies that make the updates inconvenient.

Therefore, we have been worked with Unity to enable headless (off-screen) rendering of AI2-THOR. Headless mode can be enabled by setting the `platform` to `CloudRendering` upon initialization:

```
from ai2thor.controller import Controller
from ai2thor.platform import CloudRendering
controller = Controller(platform=CloudRendering)
```

🌼 Google Colab

To get started with AI2-THOR, you may find it useful to use AI2-THOR Colab. It enables you to run AI2-THOR on Google Cloud for free in a browser, without any setup on your local device!

> **Remark**
>
> By default, AI2-THOR does not work out of the box with Google Colab, as it requires the use of Xorg for AI2-THOR to work.

🐳 Docker

For users that prefer Docker, we provide AI2-THOR Docker as a mini-framework to simplify the setup.

🎯 Requirements

OS	macOS 10.9+ or Ubuntu 14.04+
Python	Python 3.5+
CPU	SSE2 instruction set support
Graphics Card	DX9 (shader model 3.0) or DX11 with feature level 9.3 capabilities
Linux Users	X server with GLX module enabled

🤖 Agent
🛏️ Scene
🕹️ Action
🪑 Sim Object
💡 Object Visibility
🍞 Object Interactability
🥡 Receptacle

🤖 Agent

An agent is a capsule shaped entity that can navigate within scenes and interact with objects.

🛏️ Scene

A scene within AI2-THOR represents a virtual room that an agent can navigate and interact in. There are 4 scene Categories, each with 30 unique scenes within them: Kitchen, Living Room, Bedroom, Bathroom.

🕹️ Action

An action is a discrete command for the agent to perform within a scene (e.g. MoveAhead, RotateRight, PickupObject).

🪑 Sim Object

A sim object is an object that can be interacted with by the agent. Objects have a range of interactions based on their assigned `ObjectType`.

💡 Object Visibility

An object is considered `visible` when it satisfies two conditions:

1. **In frame**. The object appears in the camera's frame (i.e., it is within the camera's field of view). Even if an object is partially blocked by another object, as long as some of it is in the frame, that is sufficient for this condition.
2. **Proximity**. The object is within a distance of *visibilityDistance* from the agent's central vertical axis. By default, *visibilityDistance* is set to 1.5 meters, but it can be overwritten upon initialization.
3. **Obstruction.** The object is not obstructed. Here, a ray emitted from the camera must hit the object without first hitting another object.

> **Warning**
>
> An object rendered in an image will not always be visible to the agent. For example, an object outside the 1.5 meter threshold could be seen in an image, but will be reported as not-visible to the agent.

Each object's visibility is reported in the metadata.

🍞 Object Interactability

An object is said to be interactable if it is both `visible` and if it is unobstructed by any other objects. For instance, if a `Sponge` is behind `ShowerGlass`, then it will report `visible` as `True`, but the agent will be unable to interact with it due to the obstruction.

> **Warning**
>
> If an object is interactable, then that implies the object is visible. But, if an object is visible, that does not imply it is interactable.

🥡 Receptacle

A `Receptacle` is a type of object that may be the parent of another object. Examples include `TableTop`, `Cup`, `Sofa`, `Bed`, `Desk`, and `Bowl`. The full set of `Receptacle` object types can be filtered on the Object Types.

🤩 Initialization
🌚 Reset
⏱️ Controlling Time

🤩 Initialization

After installing AI2-THOR, we can initialize a controller, which will allow us to execute actions in the environment.

```
from ai2thor.controller import Controller

controller = Controller(
    agentMode="default",
    visibilityDistance=1.5,
    scene="FloorPlan212",

    # step sizes
    gridSize=0.25,
    snapToGrid=True,
    rotateStepDegrees=90,

    # image modalities
    renderDepthImage=False,
    renderInstanceSegmentation=False,

    # camera properties
    width=300,
    height=300,
    fieldOfView=90
)
```

Controller Parameters

agentMode: str = "default"

The type of agent that will interact with the scene. For iTHOR, stick with `agentMode="default"`. RoboTHOR provides the `locobot` as an `agentMode` option, and ManipulaTHOR provides the `arm` as an `agentMode` option.

visibilityDistance: float = 1.5

Used to set the maximum goal distance in meters between the agent and an object. This affects each object's `visible` property, where, when `True`, then the agent is within `visibilityDistance` of that object and the object appears in the agent's current egocentric frame.

scene: Optional[str] = None

The name of which scene to initialize. Valid iTHOR scenes are listed here. If unspecified and the `default` agent is being used, an iTHOR scene will be chosen at random.

gridSize: float = 0.25

The amount, in meters, that the agent moves after calling a move action (e.g., `MoveAhead` and `MoveBack`).

snapToGrid: bool = True

Determines whether the agent's position is snapped to a grid point after any movement action (e.g., `MoveAhead` and `TeleportFull`). Grid points are spaced out by `gridSize`. Set to `False` to allow diagonal movement.

rotateStepDegrees: float = 90

The default amount, in degrees, that the agent rotates after calling a rotate action (i.e., `RotateLeft` or `RotateRight`).

renderDepthImage: bool = False

When `True`, a depth frame is rendered and made available as an `event.depth_frame` attribute.

> **Remark**
> We require this to be explicitly passed in because rendering depth takes longer than rendering RGB alone.

renderInstanceSegmentation: bool = False

When `True`, a instance segmentation frame is rendered and made available as an `event.instance_segmentation_frame` attribute. Instance segmentation provides segmentation by object instance, where all objects in view are distinguishable.

> **Remark**
> We require this to be explicitly passed in because rendering instance segmentation takes longer than rendering RGB alone.

width: int = 300

The number of horizontal sampled pixels for each frame. This affects every rendered image frame (e.g., `event.frame`, `event.depth_frame`, `event.cv2img`).

height: int = 300

The number of vertical sampled pixels for each frame. This affects every rendered image frame (e.g., `event.frame`, `event.depth_frame`, `event.cv2img`).

fieldOfView: int = 90

Changes the camera's optical field of view, in degrees. Valid values are in the domain $(0:180)$. The default field of view when `agentMode="default"` is 90 degrees.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

> **Warning**
> Not all agent modes have the same default `fieldOfView`. For instance, the `locobot` has a default `fieldOfView` of 60 degrees.

🌚 Reset

Any `Controller` Initialization parameter can later be changed by calling the reset method. For instance, we can call:

```
controller.reset(scene="FloorPlan319", rotateStepDegrees=30)
```

The values will default to what they were upon the most recent reset or initialization. For instance, if you initialized with `fieldOfView=45`, then called `reset` with `fieldOfView=60`, calling `reset` again, without passing in `fieldOfView` will keep `fieldOfView=60`.

⏱️ Controlling Time

By default, AI2-THOR executes high-level actions that provide a response from the environment after the action completes execution. For instance, if the agent successfully executes an `OpenObject` action, the response is the state of the environment after the action has completely finished executing.

However, we also provide the ability to observe intermediate results, which is discussed in this section.

🐌 Pause Physics Auto-Simulation

`PausePhysicsAutoSim` toggles off auto-simulation in the environment, which must be called before one can control how much time passes in-between steps.

```
controller.step("PausePhysicsAutoSim")
```

👉 Advance Physics Step

`AdvancePhysicsStep` responds with an Event after simulating the environment for `timeStep` seconds beyond the current state. Note that `PausePhysicsAutoSim` must be called first, before calling `AdvancePhysicsStep`.

```
controller.step(
    action="AdvancePhysicsStep",
    timeStep=0.01
)
```

Advance Physics Step Parameters

timeStep: float = 0.01

The amount of time in seconds to advance the physics simulation. Valid values are in $(0:0.05]$.

🥶 Unpause Physics Auto-Simulation

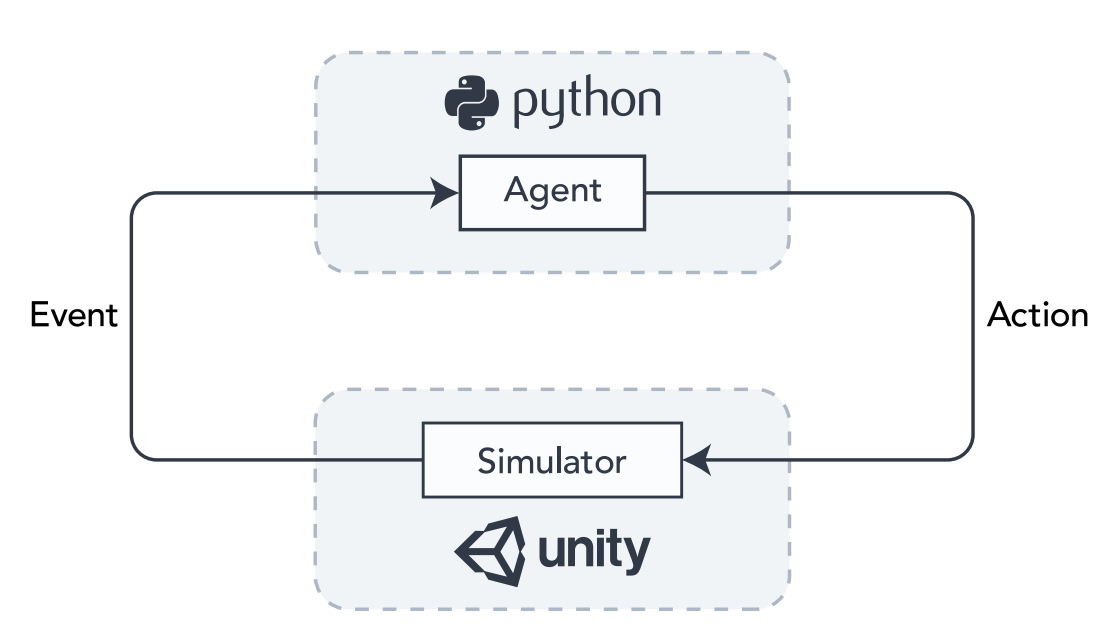
`UnpausePhysicsAutoSim` toggles back on the default physics auto-simulation, which returns future events after the action finishes execution.

> **Warning**
> Unpausing auto-simulation while objects are still in motion (i.e., `inMotion=True` on an object's metadata) will freeze each object at its current location. Thus, the remaining motion of such an object will be lost.
>
> To capture the entire movement of an object, do not unpause physics with this action until all objects return `inMotion=False` in their object metadata, which can also be checked with the shorthand of `event.metadata["isSceneAtRest"]`.

```
controller.step("UnpausePhysicsAutoSim")
```

🌏 Agent Simulator Loop
🎉 Event
🤓 Metadata
🤖 Agent Metadata
🚪 Object Metadata
🎈 Environment Queries >
🤳 Add Camera >

🌏 Agent Simulator Loop



The figure above shows AI2-THOR's agent-simulator interaction loop. Our backend scenes, actions, and observations are stored within Unity, a powerful real-time game engine. From the Python side, we sequentially interact with Unity by taking actions using AI2-THOR's Python `Controller`.

🎉 Event

The returned data after executing any action is an `Event`. Each `Event` contains a ton of information to encode the state of the environment after the action has been executed:

```
event = controller.step(...)
```

References the last executed Event

```
controller.last_event
```

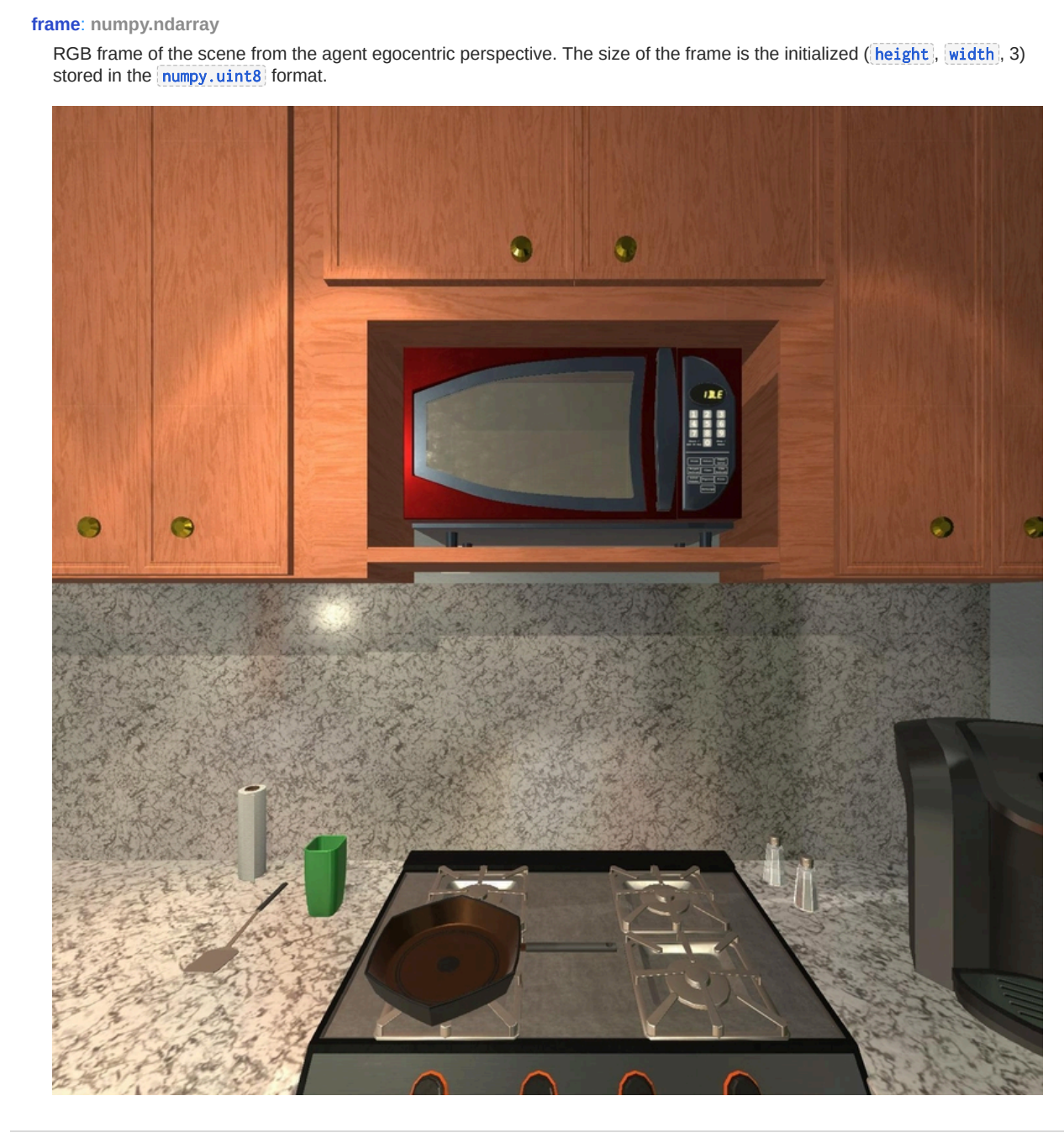
Response

```
<ai2thor.server.Event
    .metadata: {...}

    .frame: {...}
    .cv2img: {...}

    .depth_frame: {...}

    .instance_segmentation_frame: {...}
    .instance_masks: {...}
    .instance_detections2D: {...}

    .color_to_object_id: {...}
    .object_id_to_color: {...}
/>
```

Event Response

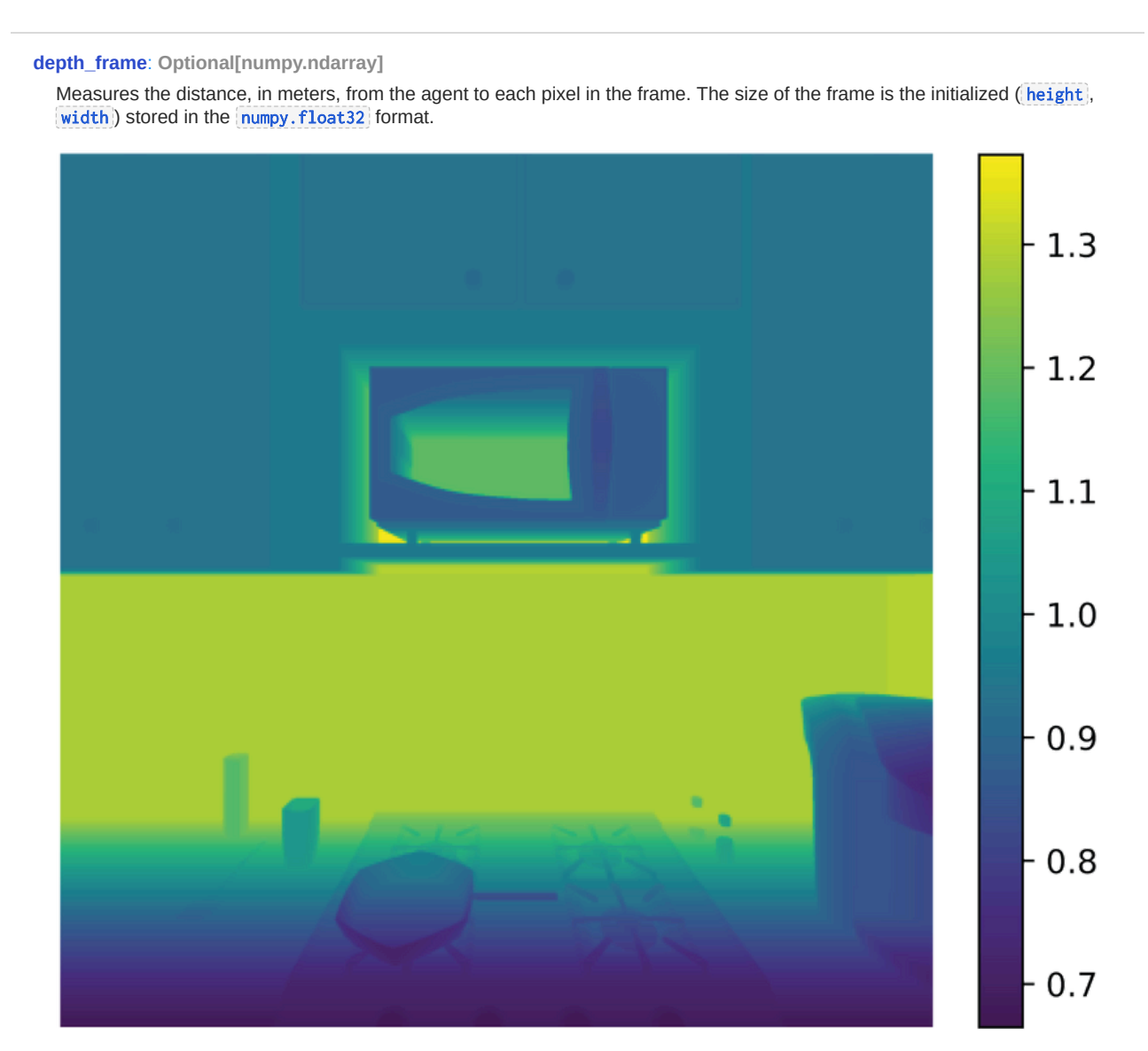
metadata: dict[str, any]
Contains information about the action and how it affected the environment. See the `Metadata` section for more.

frame: numpy.ndarray
RGB frame of the scene from the agent egocentric perspective. The size of the frame is the initialized (`height`, `width`, 3) stored in the `numpy.uint8` format.



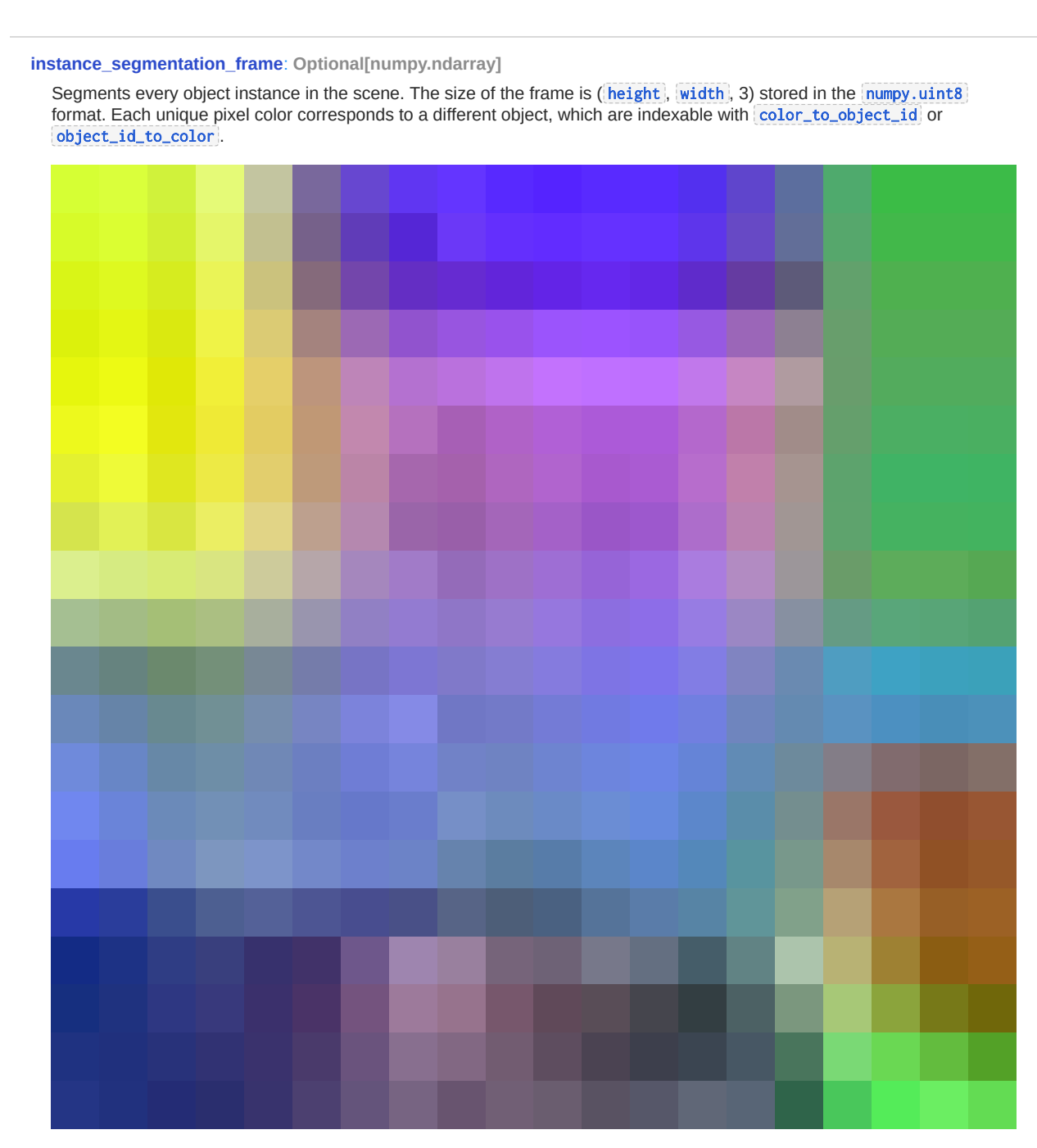
cv2img: numpy.ndarray
Same as the `frame`, except image channels are in BGR ordering. This is often useful with Python's OpenCV module (i.e., `cv2`), which expects images with BGR ordering.

depth_frame: Optional[numpy.ndarray]
Measures the distance, in meters, from the agent to each pixel in the frame. The size of the frame is the initialized (`height`, `width`) stored in the `numpy.float32` format.



> **Remark**
> Upon `initialization` or `reset`, `renderDepthImage` must be set to `True` in order for the depth frame to appear.

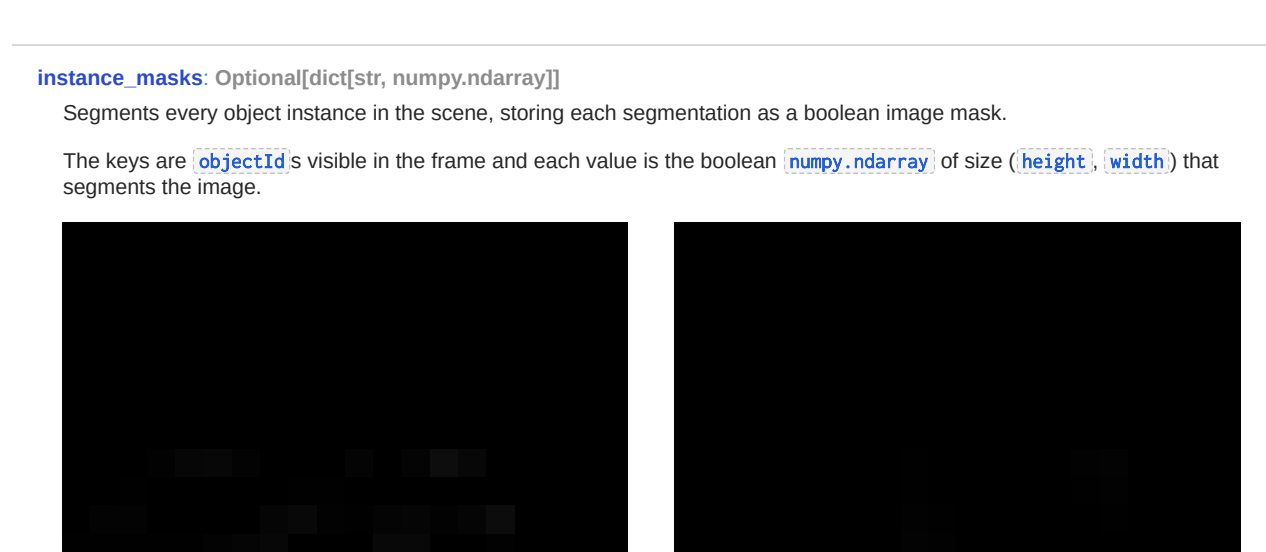
instance_segmentation_frame: Optional[numpy.ndarray]
Segments every object instance in the scene. The size of the frame is (`height`, `width`, 3) stored in the `numpy.uint8` format. Each unique pixel color corresponds to a different object, which are indexable with `color_to_object_id` or `object_id_to_color`.



> **Remark**
> Upon `initialization` or `reset`, `renderInstanceSegmentation` must be set to `True` in order for the instance segmentation frame to appear.

instance_masks: Optional[dict[str, numpy.ndarray]]
Segments every object instance in the scene, storing each segmentation as a boolean image mask.

The keys are `objectId`s visible in the frame and each value is the boolean `numpy.ndarray` of size (`height`, `width`) that segments the image.



> **Remark**
> Upon `initialization` or `reset`, `renderInstanceSegmentation` must be set to `True` in order for the instance masks to appear.

instance_detections2D: Optional[dict[str, numpy.array]]
Provides the 2D bounding box around every object instance in the scene.

The keys are object IDs and the values are [Upper Left x, Upper Left y, Lower Right x, Lower Right y], where each element is the number of pixels it is from the top left corner of the image.

> **Remark**
> Upon `initialization` or `reset`, `renderInstanceSegmentation` must be set to `True` in order for the 2D instance detections to appear.

color_to_object_id: dict[tuple[int, int, int], str]
Provides a mapping from each pixel's color, represented as a hashable tuple, in the `instance_segmentation_frame` to what object it represents.

object_id_to_color: dict[str, tuple[int, int, int]]
Same as `color_to_object_id`, except for the keys and values being swapped.

🤓 Metadata

The metadata dictionary contains summary information about the state of the environment.

```
event.metadata
```

Response

```
{
    "errorMessage": {...},
    "lastActionSuccess": {...},
    "actionReturn": {...},
    "lastAction": {...},

    "sceneName": {...},
    "sceneBounds": {...},

    "agent": {...},
    "objects": {...},

    "fov": {...},
    "screenWidth": {...},
    "screenHeight": {...},
    {...}
}
```

Metadata Response

errorMessage: str
If an action fails, feedback is provided as to what caused the failure.

lastActionSuccess: bool
States whether the last action was able to successfully execute.

An exception will not be raised upon invalid states. For instance, if an agent is standing right in-front of a wall and tries to `MoveAhead`, it will fail, but an exception will not be raised.

If an action is unsuccessful, the state of the environment will not have changed.

actionReturn: any
Holds the resulting data for actions that query the environment, such as `GetReachablePositions`.

lastAction: str
The name of the action passed into the `Controller`.

sceneName: str
The name of the scene that the agent is currently located in.

sceneBounds: dict[str, any]
This object provides all coordinates that are within bounds of the scene. This can be used in tandem with actions like `PlaceObjectAtPoint` to make sure the coordinate used is not out of bounds. This returns a `sceneBounds` object that includes an 8x3 matrix of xyz coordinates that represent the 8 corners of the box encompassing the entire scene, an xyz dictionary for the coordinates of the center of that box, and an xyz dictionary for the size (extents) of that box. Example:

```
{
  "center": {
    'x': -1.45,
    'y': 1.407,
    'z': 0.2
  },
  "cornerPoints": [
    [ 1.5,   2.92,  3.2],
    [ 1.5,   2.92, -2.8],
    [ 1.5, -0.106,  3.2],
    [ 1.5, -0.106, -2.8],
    [-4.4,   2.92,  3.2],
    [-4.4,   2.92, -2.8],
    [-4.4, -0.106,  3.2],
    [-4.4, -0.106, -2.8]
  ],
  "size": {
    "x": 5.9,
    "y": 3.02,
    "z": 6.0
  }
}
```

agent: dict[str, any]
Information about the pose of the agent. See Agent Metadata for more.

🤖 Agent Metadata

Within the metadata dictionary, the `agent` key contains the pose of the agent after the action has executed.

```
event.metadata["agent"]
```

Response

```
{
    cameraHorizon: {...},
    isStanding: {...},
    position: {...},
    rotation: {...},
    {...}
}
```

Agent Metadata Response

cameraHorizon: float
The angle in degrees that the camera's pitch is rotated.

> **Warning**
> Negative camera `horizon` values correspond to the agent looking up, whereas positive `horizon` values correspond to the agent looking down.

isStanding: bool
`True` if the agent is currently in a standing position, otherwise `False`. This bool can be changed if the agent uses the `Stand` or `Crouch` actions.

> **Warning**
> The `default` agent is currently the only agent with the ability to stand.

position: dict[str, float]
The global position of the agent, with keys for x, y, and z.

> **Remark**
> The y coordinate corresponds to upwards in 3D space.

rotation: dict[str, float]
The local rotation of the agent's body, with keys for x (pitch), y (yaw), and z (roll). Since the `default` agent's body can only change its yaw rotation, both x and z will always be approximately 0.

🚪 Object Metadata

Each object has a plethora of information exposed about it in each event.

> **Remark**
> Beyond what is shown here, the object metadata also provides information for each object state change action, which is documented on the Object State Changes page.

```
event.metadata["objects"][i]
```

Response

```
{
    "objectId": {...},
    "objectType": {...},
    "name": {...},

    "distance": {...},
    "visible": {...},

    "position": {...},
    "rotation": {...},

    "axisAlignedBoundingBox": {...},
    "objectOrientedBoundingBox": {...},

    "mass": {...},
    "salientMaterials": {...},

    "parentReceptacles": {...},
    "receptacle": {...},
    "receptacleObjectIds": {...},

    "ObjectTemperature": {...},
    "canChangeTempToHot": {...},
    "canChangeTempToCold": {...},

    "moveable": {...},
    "isMoving": {...},

    "pickupable": {...},
    "isPickedUp": {...},

    {...Object State Changes...}
}
```

Object Metadata Response

objectId: str
The unique ID of each object in the scene. It is generated at runtime and composed of an object's `objectType` and `position`.

Example: `AlarmClock|-02.08|+00.94|-03.62`.

objectType: str
The annotated type of the object. Each time is specified in the `Object Types` section.

name: str
Name of the object in Unity Scene. These names are unique within any individual scene.

distance: float
The Euclidean distance from near the center-point of the object to the agent's camera.

visible: bool
Indicates whether the object is visible, and within the initialized visibility distance of the agent.

> **Warning**
> The visible property does not mean the object is literally visible in the frame. Rather, the object both has to be visible and at a distance of less than the initialized `visibilityDistance` away.

position: dict[str, float]
The global position of the object, with keys for x, y, and z.

Note that y corresponds to the upward coordinate in 3D space.

rotation: dict[str, float]
The local rotation of the object, with keys for x (pitch), y (yaw), and z (roll).

axisAlignedBoundingBox: dict[str, union[str, any]]
Returns an axisAlignedBoundingBox object that includes an 8x3 matrix of xyz coordinates that represent the 8 corners of the box, an xyz dictionary for the coordinates of the center of the box, and an xyz dictionary for the size (extents) of the box. This axis aligned bounding box is the smallest box that can completely encloses the sim object that is aligned to the world axis. This means that if the object is rotated or moved, the overall size and volume of this axis aligned box will change in order to remain aligned relative to the static, world-axis. This is best used for rough approximations of the area/volume an object takes up. Do note that large, oddly shaped objects like CounterTops that wrap around kitchens may have awkwardly defined axis-aligned bounds. Example:

```
{
  "center": {
    "x": -1.336,
    "y":  1.098,
    "z":  0.221
  },
  "cornerPoints": [
    [-1.232, 1.277, 0.319],
    [-1.232, 1.277, 0.124],
    [-1.232, 0.919, 0.319],
    [-1.232, 0.919, 0.124],
    [ -1.44, 1.277, 0.319],
    [ -1.44, 1.277, 0.124],
    [ -1.44, 0.919, 0.319],
    [ -1.44, 0.919, 0.124],
  ],
  "size": {
    "x": 0.208,
    "y": 0.358,
    "z": 0.195
  }
}
```

objectOrientedBoundingBox: optional[dict[str, list[list[float]]]]
Returns an objectOrientedBoundingBox object that includes an 8x3 matrix of xyz coordinates that represent the 8 corners of the object oriented box. This object oriented bounding box is a box that completely encloses a sim object. The difference between this object oriented box and the axis aligned box above is this box's dimensions are static relative to the object's rotation and position. This means this object oriented box will always have the same total volume regardless of how the object is manipulated/rotated/moved. Note that only Pickupable objects have an objectOrientedBoundingBox at this time. Example:

```
{
  'cornerPoints': [
    [-1.445, 0.910, 0.115],
    [-1.228, 0.910, 0.115],
    [-1.228, 0.910, 0.328],
    [-1.445, 0.910, 0.328],
    [-1.445, 1.284, 0.115],
    [-1.228, 1.284, 0.115],
    [-1.228, 1.284, 0.328],
    [-1.445, 1.284, 0.328]
  ]
}
```

mass: float
The mass of a Pickupable or Moveable sim object in Kilograms

salientMaterials: list[str]
Array of strings listing the salient materials a pickupable object is composed of. Valid strings are: Metal, Wood, Plastic, Glass, Ceramic, Stone, Fabric, Rubber, Food, Paper, Wax, Soap, Sponge, and Organic.

parentReceptacles: list[str]
A list of objectId strings of all receptacles that contain this object.

receptacle: bool
If True, this object has the Receptacle property and can contain other objects.

receptacleObjectIds: list[str]
If the object is a receptacle, this is an array of objectIds that the receptacle contains.

ObjectTemperature: str
String that indicates what this object's current abstracted temperature is. Valid strings are: Hot, Cold, RoomTemp.

canChangeTempToHot: bool
If True, this object is a source of Heat and can contextually change other object's Temperature to Hot.

canChangeTempToCold: bool
If True, this object is a source of Cold and can contextually change other object's Temperature to Cold.

moveable: bool
If True, this object has the Moveable property and can be moved around the environment with actions like PushObject.

isMoving: bool
A bool tracking if this object is actively in motion. This can be useful when tracking object behaviors while using PausePhysicsAutoSim and AdvancePhysicsStep actions.

pickupable: bool
If True, this object has the Pickupable property and can be picked up by the agent with the PickupObject action.

isPickedUp: bool
Only used if this object is pickupable = True. If True, this object is currently being picked up by the agent. If False, the object is not being held by the agent.

🎈 Environment Queries

Environment queries are actions that query the environment to extract additional metadata regarding the current state. Since there is a performance cost that comes from calculating each environment query, they are not automatically provided in the metadata after each action.

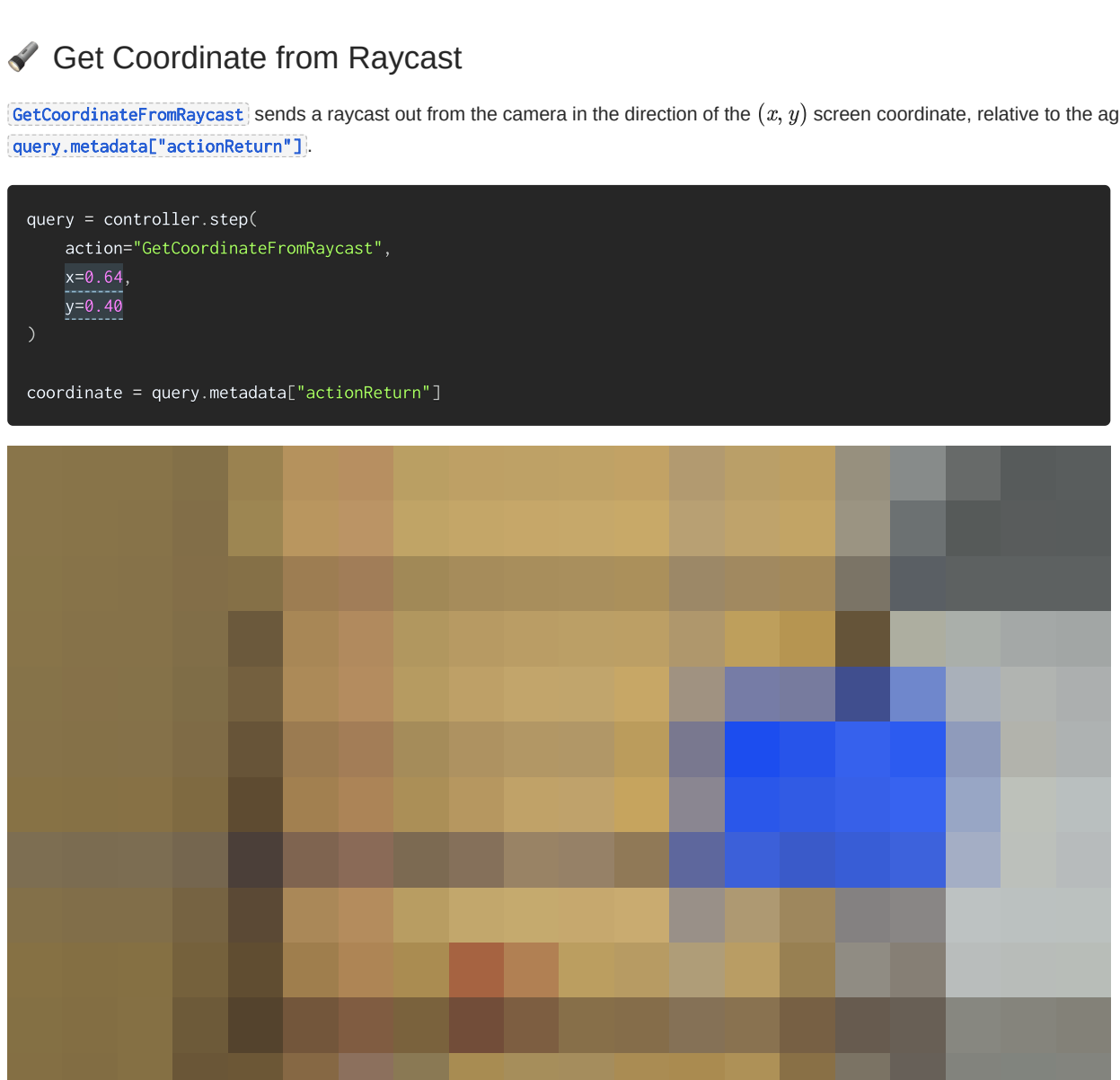
Query actions do not alter the state of the environment. Thus, they are often substantially faster than non-query actions (e.g., MoveAhead, RotateRight), since image frames and object metadata can be reused from the previous Event.

🖼️ Get Object in Frame

`GetObjectInFrame` queries the current view of the agent for the object that appears at specified (x, y) coordinate, relative to its current view. If there is an object that appears at the coordinate, its objectId is provided in `query.metadata["actionReturn"]`. Alternatively, if no object is at the provided coordinate, `query.metadata["actionReturn"]` will be `None` and `bool(query)` will be `False`.

This action can be used in tandem with object interaction actions, where `GetObjectInFrame` is first called to extract an `objectId`, and if the objectId is extracted.

```
query = controller.step(
    action="GetObjectInFrame",
    x=0.64,
    y=0.40,
    checkVisible=False
)

object_id = query.metadata["actionReturn"]
```



Get Coordinate from Raycast Parameters

x: float — required
The x coordinate from the current image frame, corresponding to the relative distance from the left of the frame. Valid values are in $[0:1]$.

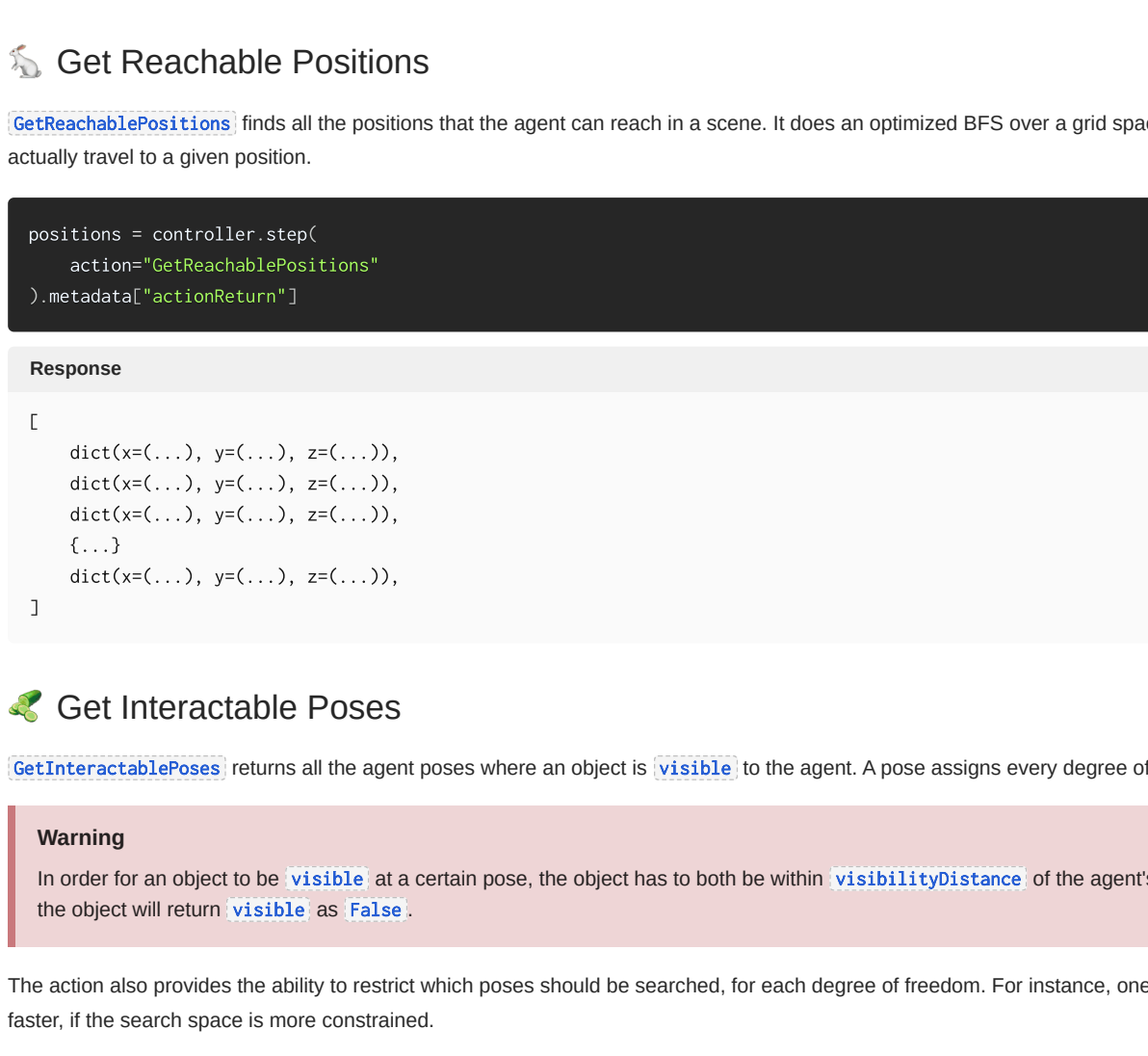
y: float — required
The y coordinate from the current image frame, corresponding to the relative distance from the top of the frame. Valid values are in $[0:1]$.

checkVisible: bool = False
If `False`, the returned object will only be provided if it is within a distance of the initialized `visibilityDistance` (default: 1.5 meters) from the agent. This is set to `False` by default so that the agent can only interact with objects in-front of it, rather than objects far across the room.

🚀 Get Coordinate from Raycast

`GetCoordinateFromRaycast` sends a raycast out from the camera in the direction of the (x, y) screen coordinate, relative to the agent's current view. The world (x, y, z) coordinate of the first point of collision that is hit on an object by the raycast is returned in `query.metadata["actionReturn"]`.

```
query = controller.step(
    action="GetCoordinateFromRaycast",
    x=0.64,
    y=0.40
)

coordinate = query.metadata["actionReturn"]
```



Get Coordinate from Raycast Parameters

x: float — required
The x coordinate from the current image frame, corresponding to the relative distance from the left of the frame. Valid values are in $[0:1]$.

y: float — required
The y coordinate from the current image frame, corresponding to the relative distance from the top of the frame. Valid values are in $[0:1]$.

🐇 Get Reachable Positions

`GetReachablePositions` finds all the positions that the agent can reach in a scene. It does an optimized BFS over a grid spaced out by the initialized `gridSize`. The valid positions are then added and returned in a list. The action can be used in tandem with Teleport, to actually travel to a given position.

```
positions = controller.step(
    action="GetReachablePositions"
).metadata["actionReturn"]
```

Response

```
[
    dict(x=(...), y=(...), z=(...)),
    dict(x=(...), y=(...), z=(...)),
    dict(x=(...), y=(...), z=(...)),
    {...}
    dict(x=(...), y=(...), z=(...)),
]
```

Get Reachable Positions Response

actionReturn: list[dict[str, float]]
A list of (x, y, z) positions that the agent can reach in the scene.

🥒 Get Interactable Poses

`GetInteractablePoses` returns all the agent poses where an object is `visible` to the agent. A pose assigns every degree of freedom on the agent to a specific value. A pose can then be passed into TeleportFull to teleport the agent to a given pose.

> **Warning**
> In order for an object to be `visible` at a certain pose, the object has to both be within `visibilityDistance` of the agent's camera and it must be in the agent's field of view. Thus, if the agent is too far away from an object, but it appears in the agent's field of view, the object will return `visible` as `False`.

The action also provides the ability to restrict which poses should be searched, for each degree of freedom. For instance, one can restrict the poses they want to poses where `horizon` is `True` and `standing` is `True`. Such restrictions can often make the action execute faster, if the search space is more constrained.

> **Warning**
> This action is expected to solely be used in tandem with TeleportFull. Hence, in future releases, if the agent is given extra degrees of freedom, those degrees of freedom will be included in each pose, and thus change the returned poses. Therefore, we recommend not indexing into any pose (i.e., `pose["horizon"]`), and instead passing the entire pose to TeleportFull using Python's `**kwargs` feature (i.e., `**pose`).

```
import numpy as np

event = controller.step(
    action="GetInteractablePoses",
    objectId="Apple|-1.0|+1.0|+1.5",
    positions=[dict(x=0, y=0.9, z=0)],
    rotations=range(0, 360, 10),
    horizons=np.linspace(-30, 60, 30),
    standings=[True, False]
)

poses = event.metadata["actionReturn"]
```

Response

```
[
    dict(
        x=(...),
        y=(...),
        z=(...),
        horizon=(...),
        rotation=(...),
        standing=(...)
    ),
    {...},
    dict(
        x=(...),
        y=(...),
        z=(...),
        horizon=(...),
        rotation=(...),
        standing=(...)
    )
]
```

TeleportFull to a Pose

```
import random
pose = random.choice(poses)

controller.step("TeleportFull", **pose)
```

Get Interactable Poses Attributes

objectId: str — required
The objectId of the object with which the interactable poses will be queried.

positions: Optional[list[dict[str, float]]] = None
Restricts which positions should be searched. If not specified, all positions from `GetReachablePositions` will be used.

rotations: Optional[list[float]] = None
Restricts which rotation values should appear in the returned response. For instance, if `[0, 180]` is passed in, only such values may appear as the rotation for each returned pose. By default, the rotation values are

$$\mathrm{range}(A_r \,\%\, D,\ \ D,\ \ 360 + A_r \,\%\, D),$$

where A_r is the current rotation of the agent and D is the initialized `rotateStepDegrees`. For instance, if $A_r = 10^\circ$ and $D = 90$, then the default rotations will be `[10, 100, 190, 280]`.

> **Warning**
> An exception is thrown if 360 mod the initialized `rotateStepDegrees` does not equal 0 and rotations has not been provided. Here, the agent cannot rotate in a circular manner, and hence, an infinite number of rotations would be possible.

horizons: Optional[list[float]] = None
Restricts which horizons should be searched. For instance, if `[0, 15]` is passed in, only such values may appear as the horizon for each returned pose. Defaults to using `[-30, 0, 30, 60]`.

> **Warning**
> Each horizon must be in $[-30:60]$.

standings: Optional[list[bool]] = None
Restricts which `standing` poses should be added to the search. For instance, if `[True]` is passed in, only values of `standing=True` may appear in the response. Defaults to `[True, False]`.

🤳 Add Camera

📷 Add Third Party Camera

`AddThirdPartyCamera` adds an invisible camera to the scene, with images available for each successive event, until reset has been called. When reset is called, the camera is removed from the scene.

```
event = controller.step(
    action="AddThirdPartyCamera",
    position=dict(x=-1.25, y=1, z=-1),
    rotation=dict(x=90, y=0, z=0),
    fieldOfView=90
)

event.third_party_camera_frames
```

Add Third Party Camera Parameters

position: dict[str, float] — required
The global (x, y, z) position of where the camera will be placed.

rotation: dict[str, float] — required
The global (x, y, z) rotation of where the camera.

fieldOfView: dict[str, float] = 90
Changes the camera's optical field of view, in degrees. Valid values are in the domain $(0:180)$.

Image frames are then accessible with the `third_party_camera_frames` attribute on successive events:

Response

```
<ai2thor.server.Event
    .third_party_camera_frames

    {...}
/>
```

Third Party Camera Response

third_party_camera_frames: list[numpy.ndarray]
A list of RGB frames from rendered from each third party camera. Images are stored as numpy.uint8 ndarrays of size (height, width, 3), where the height and width are the same as the agent's egocentric image height and width.

Camera frames in the list appear in the order that they were added to the scene.

🎥 Update Third Party Camera

`UpdateThirdPartyCamera` updates the state of a previously added third party camera that is currently in the scene. Any values that are unspecified will remain the same.

```
controller.step(
    action="UpdateThirdPartyCamera",
    thirdPartyCameraId=0,
    position=dict(x=-1.25, y=1, z=-1),
    rotation=dict(x=90, y=0, z=0),
    fieldOfView=90
)
```

Update Third Party Camera Parameters

thirdPartyCameraId: int = 0
Targets which third party camera to modify. Third party camera ids are based on the order that they were added to the scene.

Valid values are in $[0:\mathrm{len}(\text{third party cameras}) - 1]$.

position: dict[str, float]
The global (x, y, z) position of where the camera will be placed.

rotation: dict[str, float]
The global (x, y, z) rotation of how the camera will be orientated.

fieldOfView: dict[str, float]
Changes the camera's optical field of view, in degrees. Valid values are in the domain $(0:180)$.

NEW Big News! We've announced 🏘 ProcTHOR →

🛁 Scene Distribution
🚢 Scene Utility
🍌 Train/Val/Test Splits

🛁 Scene Distribution

There are 120 scenes in iTHOR, evenly spread across kitchens, living rooms, bedrooms, and bathrooms. The list of scenes is defined as:

```
kitchens = [f"FloorPlan{i}" for i in range(1, 31)]
living_rooms = [f"FloorPlan{200 + i}" for i in range(1, 31)]
bedrooms = [f"FloorPlan{300 + i}" for i in range(1, 31)]
bathrooms = [f"FloorPlan{400 + i}" for i in range(1, 31)]

scenes = kitchens + living_rooms + bedrooms + bathrooms
```

Response

```
["FloorPlan1", "FloorPlan2", "FloorPlan3", {...}, "FloorPlan430"]
```

> **Remark**
>
> Explore the scenes in the demo!

🚢 Scene Utility

We provide a cached alias to the above definitions using the `ithor_scenes` method:

```
controller.ithor_scenes(
    include_kitchens=True,
    include_living_rooms=True,
    include_bedrooms=True,
    include_bathrooms=True
)
```

Response

```
[
    "FloorPlan1",
    "FloorPlan2",
    {...},
    "FloorPlan201",
    {...},
    "FloorPlan301",
    {...},
    "FloorPlan430",
]
```

Parameters

include_kitchens: bool = True

Includes the 30 kitchens (i.e., FloorPlan[1:30]) in the returned list when True.

include_living_rooms: bool = True

Includes the 30 living rooms (i.e., FloorPlan[201:230]) in the returned list when `True`.

include_bedrooms: bool = True

Includes the 30 bedrooms (i.e., FloorPlan[301:330]) in list when `True`

include_bathrooms: bool = True

Includes the 30 bathrooms (i.e., FloorPlan[401:430]) in list when `True`.

🍌 Train/Val/Test Splits

It is common to train agents on a subset of scenes, in order to validate and test how well they generalize to new scenes and environments. For each room type, it is standard practice to treat the first 20 scenes as training scenes, the next 5 scenes as validation scenes, and the last 5 scenes as testing scenes.

Assessing Object Types

Each SimObject is assigned to an object type, which is accessible from the object's metadata:

```
from ai2thor.controller import Controller
controller = Controller(scene='FloorPlan1')

for obj in controller.last_event.metadata["objects"]:
    print(obj["objectType"])
```

iTHOR Object Types

The table below provides information about the behavior of every object type that appears in iTHOR. Definitions can be found at the bottom of the page.

Object Types that have a (*) next to them are only referenced after an interaction. For instance, `Apple` becomes `AppleSliced` once the `Slice` action has been applied to the `Apple`.

Object Type	Scenes	Actionable Properties	Material Properties	Default Compatible Receptacles	Contextual Interactions
AlarmClock	Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Box, Dresser, Desk, SideTable, DiningTable, TVStand, CoffeeTable, CounterTop, Shelf	
AluminumFoil	Kitchens (1/30)	Pickupable	Temperature, Mass, SalientMaterials		
Apple	Kitchens (All)	Pickupable, Sliceable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Microwave, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, Desk, CounterTop, GarbageCan, Dresser	
AppleSliced*	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials, Inherited: Slice (Apple)	Pot, Pan, Bowl, Microwave, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, Desk, CounterTop, GarbageCan, Dresser	
ArmChair	Living Rooms (27/30) Bedrooms (6/30)	Receptacle, Moveable	Temperature		
BaseballBat	Bedrooms (13/30)	Pickupable	Temperature, Mass, SalientMaterials	Bed, DiningTable, TVStand, CoffeeTable, SideTable, Desk, CounterTop	
BasketBall	Bedrooms (11/30)	Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Dresser, Desk, Bed, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop	
Bathtub	Bathrooms (21/30)	Receptacle (Some)	Temperature		
BathtubBasin	Bathrooms (19/30)	Receptacle	Temperature, Inherited: Bathtub		
Bed	Bedrooms (All)	Receptacle, Dirtyable	Temperature		
Blinds	Kitchens (5/30) Living Rooms (1/30) Bedrooms (27/30)	Openable	Temperature		
Book	Kitchens (2/30) Living Rooms (7/30) Bedrooms (All)	Openable, Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Box, Ottoman, Dresser, Desk, Bed, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer	
Boots	Living Rooms (5/30) Bedrooms (3/30)	Pickupable	Temperature, Mass, SalientMaterials		
Bottle	Kitchens (7/30)	Pickupable, Fillable, Breakable	Temperature, Mass, SalientMaterials	Fridge, Box, Dresser, Desk, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, GarbageCan	Will break if subjected to enough force. Will fill up with water if moved underneath a running water source.
Bowl	Kitchens (All) Living Rooms (5/30) Bedrooms (13/30)	Pickupable, Receptacle, Fillable, Breakable (Some), Dirtyable	Temperature, Mass, SalientMaterials	Microwave, Fridge, Dresser, Desk, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf	Will automatically break if subjected to enough force. Not all bowls are breakable (Plastic-Unbreakable vs Glass-Breakable vs Ceramic-Breakable). If empty, will fill up with water if moved underneath a running water source. If in the Dirty state, will automatically become Clean if moved under running water.
Box	Living Rooms (All) Bedrooms (12/30)	Openable, Pickupable, Receptacle	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Dresser, Desk, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Ottoman	
Bread	Kitchens (All)	Pickupable, Sliceable	Temperature, Mass, SalientMaterials	Microwave, Fridge, DiningTable, TVStand, CoffeeTable, SideTable, Desk, CounterTop, GarbageCan, Plate	
BreadSliced*	Kitchens (All)	Pickupable, Cookable	Temperature, Mass, SalientMaterials, Inherited: Slice (Bread)	Microwave, Fridge, DiningTable, TVStand, CoffeeTable, SideTable, Desk, CounterTop, GarbageCan, Toaster, Plate	Use PutObject action to place BreadSliced into Toaster objects. If the Toaster is on, the BreadSliced will automatically cook. BreadSliced will also automatically cook if moved over an active StoveBurner object.
ButterKnife	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Mug, Plate, Cup, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, Desk, CounterTop, Drawer	
Cabinet	Kitchens (All) Living Rooms (6/30) Bedrooms (5/30) Bathrooms (15/30)	Openable, Receptacle	Temperature		
Candle	Living Rooms (3/30) Bathrooms (All)	Pickupable, Toggleable	Temperature, Mass, SalientMaterials	Box, Dresser, Desk, Toilet, Cart, Bathtub, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer	Can be lit using the ToggleObjectOn action. If another source of flame (another Candle or a Stove Burner) touches the wick, the Candle will automatically be lit and turn on. If the wick of the Candle touches a source of water, then the Candle will automatically be put out and turned off.
CD	Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Box, Ottoman, Dresser, Desk, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan, Safe, Sofa, ArmChair	
CellPhone	Kitchens (4/30) Living Rooms (7/30) Bedrooms (All)	Pickupable, Toggleable, Breakable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Box, Ottoman, Dresser, Desk, Bed, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, Safe	Will automatically break if subjected to enough force. When broken, the CellPhone's screen will appear cracked and the On/Off state will be switched to Off. Broken CellPhones will no longer be able to turn on with ToggleObjectOn.
Chair	Kitchens (12/30) Living Rooms (20/30) Bedrooms (22/30)	Moveable	Temperature, Mass		
Cloth	Bedrooms (5/30) Bathrooms (All)	Pickupable, Dirtyable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Box, Ottoman, Dresser, LaundryHamper, Desk, Toilet, Cart, BathtubBasin, Bathtub, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	If in the Dirty state, will automatically become Clean if moved under running water.
CoffeeMachine	Kitchens (All)	Toggleable, Receptacle, Moveable	Temperature, ChangeTemp		Coffee Machines are receptacles but only allow objects of type Mug to be placed in them. If a Coffee Machine is On and an empty Mug is placed in it, the Mug will automatically fill with Coffee and have its Temperature changed to Hot
CoffeeTable	Living Rooms (24/30) Bedrooms (1/30)	Receptacle, Moveable	Temperature		
CounterTop	Kitchens (All) Bedrooms (1/30) Bathrooms (25/30)	Receptacle	Temperature		
CreditCard	Kitchens (3/30) Living Rooms (All) Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Box, Ottoman, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer, Shelf	
Cup	Kitchens (All)	Pickupable, Receptacle, Fillable, Breakable (Some), Dirtyable	Temperature, Mass, SalientMaterials	Microwave, Fridge, Dresser, Desk, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf	Will automatically break if subjected to enough force. Not all Cups are breakable (Plastic-Unbreakable vs Glass-Breakable vs Ceramic-Breakable). If empty, will fill up with water if moved underneath a running water source. If in the Dirty state, will automatically become Clean if moved under running water.
Curtains	Kitchens (2/30) Living Rooms (10/30) Bedrooms (2/30)		Temperature		
Desk	Living Rooms (2/30) Bedrooms (24/30)	Receptacle, Moveable	Temperature		
DeskLamp	Living Rooms (13/30) Bedrooms (All)	Toggleable, Moveable	Temperature		
Desktop	Bedrooms (2/30)	Moveable	Temperature, Mass, SalientMaterials		
DiningTable	Kitchens (16/30) Living Rooms (14/30)	Receptacle, Moveable	Temperature		
DishSponge	Kitchens (All) Bathrooms (6/30)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Plate, Box, Toilet, Cart, Cart, BathtubBasin, Bathtub, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
DogBed	Living Rooms (2/30) Bedrooms (1/30)	Moveable	Temperature, Mass, SalientMaterials		
Drawer	Kitchens (28/30) Living Rooms (23/30) Bedrooms (29/30) Bathrooms (15/30)	Openable, Receptacle	Temperature		
Dresser	Living Rooms (10/30) Bedrooms (11/30) Bathrooms (2/30)	Receptacle, Moveable	Temperature		
Dumbbell	Bedrooms (4/30)	Pickupable	Temperature, Mass, SalientMaterials		
Egg	Kitchens (All)	Pickupable, Sliceable, Breakable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Microwave, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, GarbageCan	Using the Slice action or the Break action will change the Egg to an EggCracked object. Dropping or Throwing the Egg with enough force will also automatically Break the egg into an EggCracked.
EggCracked*	Kitchens (All)	Pickupable, Cookable	Temperature, Mass, SalientMaterials, Inherited: Slice (Egg), Inherited: Break (Egg)	Pot, Pan, Bowl, Microwave, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, GarbageCan	Placing an EggCracked object onto an active StoveBurner will automatically cook it. Placing an EggCracked object into a Microwave, and then turning on the Microwave will automatically cook it.
Faucet	Kitchens (All) Bathrooms (All)	Toggleable	Temperature		Can be turned on with ToggleObjectOn action. When on, a stream of running water will pour out from the faucet spout. If the Faucet is over a Sink or a Bathtub, the interior of the Sink/Bathtub will fill with standing water. For objects that can be automatically cleaned, the stream of water is a Running Water Source
Floor	Kitchens (All) Living Rooms (All) Bedrooms (All) Bathrooms (All)		Temperature		
FloorLamp	Living Rooms (All)	Toggleable, Moveable	Temperature		
Footstool	Bedrooms (1/30) Bathrooms (1/30)	Moveable	Temperature		
Fork	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Mug, Plate, Cup, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer	
Fridge	Kitchens (All)	Openable, Receptacle	Temperature, ChangeTemp		Objects placed in a Fridge will have their temperature automatically changed to Cold
GarbageBag	Kitchens (3/30) Bedrooms (3/30)	Moveable	Temperature, Mass, SalientMaterials		
GarbageCan	Kitchens (All) Living Rooms (All) Bedrooms (All) Bathrooms (All)	Receptacle, Moveable	Temperature		
HandTowel	Bathrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	HandTowelHolder	
HandTowelHolder	Bathrooms (All)	Receptacle	Temperature		HandTowelHolders are recptacle objects, but only allow HandTowel type objects to be placed on them.
HousePlant	Kitchens (10/30) Living Rooms (All) Bedrooms (7/30) Bathrooms (1/30)	Fillable, Moveable	Temperature		Will fill up with water if moved underneath a running water source.
Kettle	Kitchens (15/30)	Openable, Pickupable, Fillable	Temperature, Mass, SalientMaterials	DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Sink, SinkBasin, Cabinet, StoveBurner, Shelf	Will fill up with water if moved underneath a running water source.
KeyChain	Living Rooms (All) Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Box, Ottoman, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, Safe	
Knife	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Mug, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer	
Ladle	Kitchens (10/30)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Plate, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer	
Laptop	Living Rooms (All) Bedrooms (All)	Openable, Pickupable, Toggleable, Breakable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Ottoman, Dresser, Desk, Bed, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop	Will automatically break if subjected to enough force. When broken, the Laptop's screen will appear cracked and the On/Off state will be switched to Off. Broken Laptops will no longer be able to turn on with ToggleObjectOn, but they can still be opened or closed with OpenObject or CloseObject.
LaundryHamper	Bedrooms (8/30)	Receptacle, Moveable	Temperature		
Lettuce	Kitchens (All)	Pickupable, Sliceable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, GarbageCan	
LettuceSliced*	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials, Inherited: Slice (Lettuce)	Pot, Pan, Bowl, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, GarbageCan	
LightSwitch	Kitchens (All) Living Rooms (All) Bedrooms (All) Bathrooms (All)	Toggleable	Temperature		
Microwave	Kitchens (All)	Openable, Toggleable, Receptacle, Moveable	Temperature, ChangeTemp		Any objects inside a Microwave while it is turned On will have their temperature changed to Hot. Some object will also automatically cook (Potato, Egg). Microwaves cannot be turned on if they are open, and cannot be opened if they are currently on.
Mirror	Kitchens (2/30) Living Rooms (3/30) Bedrooms (All) Bathrooms (All)	Breakable, Dirtyable	Temperature		Automatically breaks if subjected to enough force. For example, if an object thrown at it has enough force, the object may break.
Mug	Kitchens (All) Bedrooms (16/30)	Pickupable, Receptacle, Fillable, Breakable, Dirtyable	Temperature, Mass, SalientMaterials	SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf	Will automatically break if subjected to enough force. Not all Mugs are breakable (Plastic-Unbreakable vs Glass-Breakable vs Ceramic-Breakable). If empty, will fill up with water if moved underneath a running water source. If in the Dirty state, will automatically become Clean if moved under running water. If empty and placed in a CoffeeMachine that is on, will automatically fill with Coffee and have temperature changed to Hot
Newspaper	Living Rooms (18/30)	Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Ottoman, Dresser, Desk, Bed, Toilet, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
Ottoman	Living Rooms (4/30)	Receptacle, Moveable	Temperature		
Painting	Living Rooms (All) Bedrooms (13/30) Bathrooms (1/30)		Temperature		
Pan	Kitchens (All)	Pickupable, Receptacle, Dirtyable	Temperature, Mass, SalientMaterials	DiningTable, CounterTop, TVStand, CoffeeTable, SideTable, Sink, SinkBasin, Cabinet, StoveBurner, Fridge	If in the Dirty state, will automatically become Clean if moved under running water.
PaperTowelRoll	Kitchens (10/30) Bathrooms (4/30)	Pickupable, UsedUp	Temperature, Mass, SalientMaterials		
Pen	Kitchens (2/30) Living Rooms (5/30) Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Mug, Box, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
Pencil	Kitchens (2/30) Living Rooms (4/30) Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Mug, Box, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
PepperShaker	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer, Cabinet, Shelf	
Pillow	Living Rooms (All) Bedrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Ottoman, Bed	
Plate	Kitchens (All) Living Rooms (10/30)	Pickupable, Receptacle, Breakable (Some), Dirtyable	Temperature, Mass, SalientMaterials	Microwave, Fridge, Dresser, Desk, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf	Will automatically break if subjected to enough force. Not all Plates are breakable (Plastic-Unbreakable vs Glass-Breakable vs Ceramic-Breakable). If in the Dirty state, will automatically become Clean if moved under running water.
Plunger	Bathrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Cart, Cabinet	
Poster	Bedrooms (6/30)		Temperature		
Pot	Kitchens (All)	Pickupable, Receptacle, Fillable, Dirtyable	Temperature, Mass, SalientMaterials	StoveBurner, Fridge, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf	If empty, will fill up with water if moved underneath a running water source. If in the Dirty state, will automatically become Clean if moved under running water.
Potato	Kitchens (All)	Pickupable, Sliceable, Cookable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Microwave, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, GarbageCan	Will automatically be Cooked if placed over an active Stove Burner or in a turned on Microwave. When Sliced, will create multiple PotatoSliced objects. Cooking the Potato first and then Slicing it will result in Cooked PotatoSliced objects.
PotatoSliced*	Kitchens (All)	Pickupable, Cookable	Temperature, Mass, SalientMaterials, Inherited: Slice (Potato)	Pot, Pan, Bowl, Microwave, Fridge, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, GarbageCan	Will automatically be Cooked if placed over an active Stove Burner or in a turned on Microwave.
RemoteControl	Living Rooms (All) Bedrooms (3/30)	Pickupable	Temperature, Mass, SalientMaterials	Sofa, ArmChair, Box, Ottoman, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer	
RoomDecor	Living Rooms (2/30)	Moveable	Temperature, Mass, SalientMaterials		
Safe	Kitchens (1/30) Living Rooms (2/30) Bedrooms (5/30)	Openable, Receptacle, Moveable	Temperature		
SaltShaker	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer, Cabinet, Shelf	
ScrubBrush	Bathrooms (All)	Pickupable	Temperature, Mass, SalientMaterials		
Shelf	Kitchens (9/30) Living Rooms (16/30) Bedrooms (23/30) Bathrooms (7/30)	Receptacle	Temperature		
ShelvingUnit	Kitchens (5/30) Living Rooms (5/30) Bedrooms (7/30)	Moveable	Temperature, Mass, SalientMaterials		
ShowerCurtain	Bathrooms (13/30)	Openable	Temperature		
ShowerDoor	Bathrooms (17/30)	Openable, Breakable	Temperature		Automatically breaks if subjected to enough force. For example, if an object thrown at it has enough force, the object may break.
ShowerGlass	Bathrooms (13/30)	Breakable	Temperature		Automatically breaks if subjected to enough force. For example, if an object thrown at it has enough force, the object may break.
ShowerHead	Bathrooms (24/30)	Toggleable	Temperature		Can be turned on with ToggleObjectOn action. When on, a stream of running water will pour out from the ShowerHead. For objects that can be automatically cleaned, the stream of water is a Running Water Source.
SideTable	Kitchens (3/30) Living Rooms (27/30) Bedrooms (23/30) Bathrooms (5/30)	Receptacle, Moveable	Temperature		
Sink	Kitchens (All) Bathrooms (All)	Receptacle (Some)	Temperature		
SinkBasin	Kitchens (All) Bathrooms (All)	Receptacle	Temperature, Inherited: Sink		
SoapBar	Bathrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Toilet, Cart, Bathtub, BathtubBasin, Sink, SinkBasin, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
SoapBottle	Kitchens (All) Bathrooms (All)	Pickupable, UsedUp (Some)	Temperature, Mass, SalientMaterials	Dresser, Desk, Toilet, Cart, Bathtub, Sink, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
Sofa	Living Rooms (All) Bedrooms (1/30)	Receptacle, Moveable	Temperature		
Spatula	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Plate, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer	
Spoon	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials	Pot, Pan, Bowl, Mug, Plate, Cup, Sink, SinkBasin, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Drawer	
SprayBottle	Kitchens (2/30) Bathrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	Dresser, Desk, Toilet, Cart, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
Statue	Kitchens (4/30) Living Rooms (29/30) Bedrooms (4/30)	Pickupable, Breakable	Temperature, Mass, SalientMaterials	Box, Dresser, Desk, Cart, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Safe	Will automatically break if subjected to enough force, for example if dropped from a high enough distance
Stool	Kitchens (14/30) Living Rooms (2/30) Bedrooms (1/30)	Moveable	Temperature, Mass, SalientMaterials		
StoveBurner	Kitchens (All)	Toggleable, Receptacle	Temperature, ChangeTemp		While this object type can be turned on/off, the ToggleObjectOn and ToggleObjectOff actions will not work on it directly. All StoveBurners are controlled by the on/off state of a StoveKnob object. Turning on/off the StoveKnob will automatically turn on/off the StoveBurner that it is linked to. Objects placed over an active StoveBurner will have their temperature changed to Hot. Some objects can be automatically cooked over an active StoveBurner (Potato, BreadSliced). StoveBurners are receptacles but only allow objects of type Pot, Pan, and Kettle to be placed on them directly. StoveBurners are also sources of fire and can light Candles if the wick of the candle is moved over the burner
StoveKnob	Kitchens (All)	Toggleable	Temperature		All StoveKnobs control the on/off state of themselves and a linked StoveBurner.
TableTopDecor	Bedrooms (1/30)	Pickupable	Temperature, Mass, SalientMaterials		
TargetCircle			Temperature		Can only be spawned in via the `SpawnTargetCircles` action (See the Miscellaneous Actions section). `TargetCircles` are a special type of sim object that have no collision, but do return what objects are contained by the circle in the circle's object metadata. This can be used to create target areas to maneuver other sim objects to, allowing for a success or failure flag to see if an object was succesfully moved to a specific target circle's position. There are three sizes of target circles: small, medium, and large. This allows for easier or harder target areas.
TeddyBear	Bedrooms (10/30)	Pickupable	Temperature, Mass, SalientMaterials	Bed, Sofa, ArmChair, Ottoman, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Safe	
Television	Living Rooms (All) Bedrooms (3/30)	Toggleable, Breakable, Moveable	Temperature		Will automatically break if subjected to enough force. When broken, the Television's screen will appear cracked and the On/Off state will be switched to Off. Broken Televisions will no longer be able to turn on with ToggleObjectOn.
TennisRacket	Bedrooms (10/30)	Pickupable	Temperature, Mass, SalientMaterials	Dresser, Desk, Bed, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop	
TissueBox	Living Rooms (9/30) Bedrooms (4/30) Bathrooms (9/30)	Pickupable, UsedUp (Some)	Temperature, Mass, SalientMaterials	Box, Dresser, Desk, Toilet, Cart, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
Toaster	Kitchens (All)	Toggleable, Receptacle, Moveable	Temperature, ChangeTemp		Toasters are receptacles but only allow BreadSliced objects to be placed in them. Any BreadSliced object in a Toaster that is turned on will automatically be Cooked.
Toilet	Bathrooms (All)	Openable, Receptacle	Temperature		
ToiletPaper	Bathrooms (All)	Pickupable, UsedUp	Temperature, Mass, SalientMaterials	Dresser, Desk, Toilet, ToiletPaperHanger, Cart, Bathtub, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, GarbageCan	
ToiletPaperHanger	Bathrooms (All)	Receptacle	Temperature		ToiletPaperHangers are receptacles but only allow ToiletPaper objects to be placed in them
Tomato	Kitchens (All)	Pickupable, Sliceable	Temperature, Mass, SalientMaterials	DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Sink, SinkBasin, Pot, Bowl, Fridge, GarbageCan, Plate	When Sliced, will create multiple TomatoSliced objects.
TomatoSliced*	Kitchens (All)	Pickupable	Temperature, Mass, SalientMaterials, Inherited: Slice (Tomato)	DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Sink, SinkBasin, Pot, Bowl, Fridge, GarbageCan, Plate	
Towel	Bathrooms (All)	Pickupable	Temperature, Mass, SalientMaterials	TowelHolder	
TowelHolder	Bathrooms (All)	Receptacle	Temperature		TowelHolders are receptacles but only allow Towel objects to be placed in them.
TVStand	Living Rooms (12/30)	Receptacle, Moveable	Temperature		
VacuumCleaner	Bedrooms (1/30)	Moveable	Temperature, Mass, SalientMaterials		
Vase	Kitchens (6/30) Living Rooms (18/30) Bedrooms (3/30)	Pickupable, Breakable	Temperature, Mass, SalientMaterials	Box, Dresser, Desk, Cart, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Safe	Will automaticaly break if subjected to enough force.
Watch	Living Rooms (Some)	Pickupable	Temperature, Mass, SalientMaterials	Box, Dresser, Desk, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer, Safe	
WateringCan	Living Rooms (13/30)	Pickupable, Fillable	Temperature, Mass, SalientMaterials	Dresser, Desk, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, Drawer	If empty, will automatically fill with water if placed under a running water source.
Window	Kitchens (26/30) Living Rooms (28/30) Bedrooms (All) Bathrooms (19/30)	Breakable	Temperature		Will automatically break if subjected to enough force.
WineBottle	Kitchens (10/30)	Pickupable, Fillable, Breakable	Temperature, Mass, SalientMaterials	Fridge, Dresser, Desk, Cabinet, DiningTable, TVStand, CoffeeTable, SideTable, CounterTop, Shelf, GarbageCan	Will automatically break if subjected to enough force. If empty, will automatically fill with water if placed under a running water source.

Definitions

Actionable Properties

Objects in this framework can have a number of Actionable Properties associated with the object type. Actionable Properties are specific properties that have an Action associated with them. For example, an object that is Openable means that the OpenObject and CloseObject actions can be used to interact with that object. Below is a list of all Actionable Properties and their detailed descriptions.

Openable

These objects can be opened or closed using the OpenObject and CloseObject actions. Receptacles that are Openable allow for objects to be placed inside them.

All Openable objects will return openable=True in their object metadata to indicate that they have this property. The object metadata also indicates an Openable object's current open/closed state. If isOpen=True, the object is open. If isOpen=False, the object is closed.

Pickupable

These objects are able to be picked up or put down into Receptacles objects by the Agent using the PickupObject and PutObject actions. Picked up objects can also be dropped using DropHandObject which will remove the object from the Agent's hand without needing a target receptacle. Throw is an extension of dropping an object, where an additional force is added to throw object about the scene. Pickupable objects can also be shoved around using the Push and Pull actions. Some Receptacle objects are also pickupable. Any objects placed inside a pickupable receptacle are moved all at once, so complex sequences like (Put Apple on Plate, then Move Plate with Apple to Sink) are possible.

All Pickupable objects will return pickupable=True in their object metadata to indicate that they have this property. The object metadata also indicates a Pickupable object's current state of whether it is currently picked up by the agent or not. If isPickedUp=True the agent is actively holding the object. If isPickedUp=False then the object is not being picked up by the agent.

Moveable

These are non-static objects that can be moved around the scene by using actions like Push and Pull. They can also be repositioned due to forces from other objects, such as using Throw to toss a Pickupable object at a Moveable object. The key difference between Moveable and Pickupable objects are that Moveable objects are too large to be held in the agent's hand as Pickupable objects are, so actions like PickupObject will not work on Moveable objects.

All Moveable objects will return moveable=True in their object metadata to indicate that they have this property. The object metadata also indicates if an object is currently in motion. If isMoving=True then the object is actively moving. If isMoving = False the object is not moving. Note that both Moveable and Pickupable objects can be moving, so the isMoving metadata value is not restricted to only Moveable objects.

Toggleable

Interact with these objects using the ToggleObjectOn and ToggleObjectOff actions. All Toggleable objects have a visible state change that occurs when toggled on or off (i.e., laptop screen will be on with an image or blank when off, a lamp will be lit when on and dim when off).

All Toggleable objects will return toggleable=True in their object metadata to indicate that they have this property. The object metadata also indicates if a Toggleable object is currently toggled On or Off. If isToggled=True, the object is turned on. If isToggled=False, the object is turned off.

Receptacle

Receptacle objects allow other objects to be placed on or in them if the other object can physically fit the receptacle. Some receptacles are restricted to specific object types (i.e. ToiletPaper is the only type that can go on a ToiletPaperHanger). Note that any receptacles that are also Pickupable can be moved about the scene with any sim objects they actively contain (ie: Pick up a Plate with an Apple on it to move both the Plate and Apple around the scene simultaneously).

All Receptacle objects will return receptacle=True in their object metadata to indicate they have this property. The object metadata also indicates if a Receptacle object contains any other objects via the receptacleObjectIds metadata string list, which lists all objectIds of any sim objects contained by the Receptacle. Additionally, objects report back any receptacles that they are contained by via the parentReceptacles list of strings in the object metadata.

Fillable

These objects can be filled with various liquids using the FillObjectWithLiquid action. Fillable objects can be filled with Water, Coffee, or Wine. If an object is filled with one of the liquid types and is rotated greater than 90 degrees from the global upward axis, the object will automatically empty itself because the liquid has "spilled." Additionally, some fill interactions are context sensitive. For example, placing an empty mug in a coffee maker object that is turned on will automatically fill the mug with coffee. Similarly, moving an empty mug under a running faucet will automatically fill the mug with water.

All Fillable objects will return canFillWithLiquid=True in their object metadata to indicate they have this property. The object metadata also indicates if an object is actively filled with a liquid. If isFilledWithLiquid=True the object is filled by some liquid. If isFilledWithLiquid=False, the object is empty.

Sliceable

These objects can be sliced into smaller pieces using the SliceObject action. This destroys the source object and spawns in multiple "sliced" pieces of the source object in the exact same location. This is a one-way state change, so only a scene reset will revert sliced objects to their whole versions. Sliced objects will still report metadata information even after being destroyed. This allows you to check the last position the source object was before the Slice action finished.

All Sliceable objects will return sliceable=True in their object metadata to indicate if they have this property. The object metadata also indicates if a Sliceable object has been sliced or not. If isSliced=True the object has been sliced and is no longer interactable (although the pieces it spawns are interactable). If isSliced=False the object has not been sliced and is whole.

Cookable

These objects have a cooked state that can be switched to with the CookObject action. This is a one-way state change, so only a scene reset will revert cooked objects back to their uncooked state. Cookable objects can interact with objects that are heat sources (canChangeTempToHot=True) to automatically change the object to a cooked state. This means interactions like turning on a Microwave with a Potato in it will turn the Potato to the cooked state without using the CookObject action.

All Cookable objects will return cookable=True in their object metadata to indicate that they have this property. The object metadata also indicates if a Cookable object is currently cooked. If isCooked=True, the object is cooked. If isCooked=False, the object is not cooked.

Breakable

These objects have a broken state that can be switched to with the BreakObject action. This is a one-way state change, so only a scene reset will revert broken objects to their unbroken state. Breakable objects also break automatically if they collide with a high enough force. This force threshold is different between objects because some objects are more fragile than others. This automatic breaking can be toggled off via the MakeObjectsOfTypeUnbreakable action. When some objects break, they will shatter into pieces that will not be interacatble, as the fractured remains of the object are not themselves sim objects. Objects that can shatter like certain Cup or Vase objects made of glass or ceramic will shatter in this way when broken. Other objects may remain interactable in a limited way after they break. A Laptop, for example, when broken will have its screen become cracked. You will still be able to open, close, and pickup the Laptop if it is broken, but you will no longer be able to toggle it on.

All Breakable objects will return breakable=True in their object metadata to indicate that they have this property. The object metadata also indicates if a Breakable object is currently broken. If isBroken=True, the object has broken. If isBroken=False, the object is whole and not broken.

Dirtyable

These objects have a clean and Dirtyable state that can be toggled between using the DirtyObject action. This includes objects like Mugs that can have grime on them, or a Bed that has the covers messy or made. Certain Dirtyable objects can contextually be switched to their clean state if they are moved under running water. (a grimy Bowl moved under a Faucet that is toggled on will clean the bowl automatically).

All Dirtyable objects will return dirtyable=True in their object metadata to indicate that they have this property. The object metadata also indicates if a Dirtyable object is currently in the dirty or clean state. If isDirty=True, the object is in its dirty state. if isDirty=False, the object is in its clean state.

UsedUp

These objects can have parts of themselves used up with the UseUpObject action. This is a one-way interaction and can not be reversed unless the scene is reset. This can change the overall look of the object to show that its contents have been "Used Up." ToiletPaper, TissueBox, and PaperTowelRoll objects are examples of objects that, when used upp, change form in some way.

All UsedUp objects will return canBeUsedUp=True in their object metadata to indicate that they have this property. The object metadata also indicates if a UsedUp object is currently used up or full. If isUsedUp=True, the object has been used up. If isUsedUp=False, the object is full.

Material Properties

Material Properties are properties of objects that don't have an Action directly associated with them. These properties cannot be directly manipulated- they only update based on contextual interactions with other sim objects or the environment.

Temperature

All objects have an abstracted Temperature value that can be either Hot, Cold, or Room Temperature.

ChangeTemp

These objects can change the Temperature value (Hot, Cold, Room Temp) of other sim objects depending on if the ChangeTemp object is a Heat or Cold source. For example, objects placed on a turned on Stove Burner will have their temperature changed to Hot automatically. Similarly, objects placed in a Refrigerator object will have their temperature changed to Cold automatically. Objects removed from either a Hot or Cold source will return to Room Temperature over time.

Mass

All Pickupable objects have a mass value in Kilograms. Using actions like Throw will realistically cause behavior that is reliant on different mass (i.e., Throwing a 0.1kg object with 100 newtons of force will fly farther than Throwing a 200kg object with 100 newtons of force). Note the mass of objects that spawn from other objects is persistent (i.e., A source Potato that has a mass of 0.5 kg, when Sliced would result in slices that add up to 0.5 kg total). Note that static objects like counter-tops or cabinets that are built into the "structure" of a scene do not have a Mass value that can be manipulated.

SalientMaterials

All Pickupable objects have a set of Salient Materials which describe what the object is made up of. Salient materials include: Metal, Wood, Plastic, Glass, Ceramic, Stone, Fabric, Rubber, Food, Paper, Wax, Soap, Sponge, Organic. Check any object's Metadata to see what Salient Materials it has, as objects in the same object type category can vary in composition (i.e., Bowls can be Plastic, Glass, or Ceramic).

Inherited

Some sim objects are created from a source sim object after certain actions. For example, PotatoSliced objects are generated after a Potato object has had the Slice action used on it. Inherited objects inerit the objectId of their source object. So if Potato|1|1|1| was sliced, it would result in multiple slices Potato|1|1|1|Slice_1, Potato|1|1|1|Slice_2, etc.

Some objects can also be Inherited if they are intrinsically a part of another object. An example of this would be every Sink object has a SinkBasin object within it. They inherit objectIds from their "parent" object similar to inherited objects that result from actions (i.e., Sink|1|2|3| would contain a SinkBasin sim object called Sink|1|2|3|SinkBasin). This is to differentiate interacting with the Sink or the interior basin of the sink, as sometimes objects want to be placed on the edge/surface of a Sink instead of the actual basin.

Note that the below table lists the Action used and original object type that results in the Inherited object type if the Inherited type is dependant on an action: PotatoSliced results from Slice(Potato). Other Object Types, like the before mentioned SinkBasin, do not require an action are are listed with just their source object that they are a part of: Bathtubs contain a BathtubBasin.

Contextual Interactions

Some objects are able to contextually affect the state or behavior of other objects. For example, turning on a Microwave with a Potato object in it will cause the Potato's metadata to automatically change the members isCooked to True and ObjectTemperature to Hot. Additionally, some objects might have certain restricted interactions with others. ToiletPaperHanger objects, which are receptacles, can only have ToiletPaper objects placed in them for example.

© The Allen Institute for AI - All Rights Reserved
Terms of Use | Privacy Policy

NEW Big News! We've announced 🏘️ ProcTHOR →

🏵️ Set Object Poses
🏋️ Set Mass Properties
🌡️ Set Temperature >
👻 Hiding Objects >

🏵️ Set Object Poses

`SetObjectPoses` is useful for setting a scene to the exact state of a previous initialization without relying on the Object Position Randomization's random seed. Use a previous metadata dump of SimObjects to get position/rotation data for each SimObject. Sets up a scene according to the provided objects and poses. Objects which are specified twice will be copied.

> **Warning**
>
> All moveable and pickupable objects will be removed from the scene if they are not specified.

```
controller.step(
  action='SetObjectPoses',
  objectPoses=[
    {
      "objectName": "Alarm_Clock_19",
      "rotation": {
        "y": 270,
        "x": 0,
        "z": 0
      },
      "position": {
        "y": 0.8197357,
        "x": 2.45610785,
        "z": 0.054755792
      }
    },
    {...},
    {
      "objectName": "Alarm_Clock_19",
      "rotation": {
        "y": 270,
        "x": 0,
        "z": 0
      },
      "position": {
        "y": 0.8197357,
        "x": 2.64169645,
        "z": -0.134690791
      }
    }
  ]
)
```

Set Object Poses Parameters

objectPoses: list[dict[str, any]] **required**

List of object names and their poses. This information can be dumped from the metadata of a prior episode. Each pose must contain keys for

- **objectName**: The name of the Sim Object reported by the object metadata's `name` parameter.
- **position**: Global coordinate position of the object, as reported in the object's `position` metadata.
- **rotation**: Local coordinate rotation of the object, as reported in the object's `rotation` metadata.

🏋️ Set Mass Properties

`SetMassProperties` changes the mass properties of any `Pickupable` or `Moveable` object.

```
controller.step(
    action="SetMassProperties",
    objectId="Apple|+1.25|+0.25|-0.75",
    mass=22.5,
    drag=15.9,
    angularDrag=5.6
)
```

Set Mass Properties Parameters

objectId: str **required**

The unique identifier of a `Moveable` object, found in the object metadata.

mass: float **required**

The new object's mass, in kilograms. Must be greater than 0.

drag: float **required**

The new drag coefficient of the object, which determines its resistance when it is in motion. Higher values slow the object down more. Must be greater than 0.

angularDrag: float **required**

The new angular drag coefficient of the object, which determines its resistance when it is in rotational motion. Higher values make it harder to rotate the object. Must be greater than 0.

🌡️ Set Temperature

🍳 Object Type Temperature Decay Time

`SetRoomTempDecayTimeForType` changes all objects specified by objectType in the current scene to have a new Room Temp Decay Time specified by TimeUntilRoomTemp. This can be done to adjust the amount of time it takes for specific object types to return to room temperature. By default, all objects begin at Room Temperature. Objects placed on/in Hot or Cold sources (stove burner, microwave, fridge etc.) will have their ObjectTemperature value changed to Hot or Cold. If the object is removed from a Hot or Cold source, they will return to room temperature over time. The default time it takes for an object to decay to Room Temp is 10 seconds.

```
controller.step(
    action="SetRoomTempDecayTimeForType",
    objectType="Bread",
    TimeUntilRoomTemp=20.0
)
```

Parameters

objectType: str

The object type to change the decay timer of. See a full list of Object Types on the Object Types page.

TimeUntilRoomTemp: float **required**

The amount of time it will take for an object to decay from Hot/Cold to Room Temp.

🌎 Global Temperature Decay Time

`SetGlobalRoomTempDecayTime` changes all objects in the current scene to have a new Room Temp Decay Time specified by TimeUntilRoomTemp. By default, all objects begin at Room Temperature. Objects placed on/in Hot or Cold sources (stove burner, microwave, fridge etc.) will have their ObjectTemperature value changed to Hot or Cold. If the object is removed from a Hot or Cold source, they will return to room temperature over time. The default time it takes for an object to decay to Room Temp is 10 seconds.

```
controller.step(
    action="SetGlobalRoomTempDecayTime",
    TimeUntilRoomTemp=20.0
)
```

Parameters

TimeUntilRoomTemp: float **required**

The amount of time it will take for an object to decay from Hot/Cold to Room Temp.

☕ Disable Temperature Decay

`SetDecayTemperatureBool` disables the decay over time of the ObjectTemperature of objects. If set to False, objects will not decay to room temperature and will remain Hot/Cold even if removed from the Hot/Cold source.

```
controller.step(
    action='SetDecayTemperatureBool',
    allowDecayTemperature=False
)
```

allowDecayTemperature: bool = True

Set to allow object Temperatures to decay over time.

👻 Hiding Objects

> **Warning**
>
> Hiding an object may cause unintended interactions with other objects after an object is disabled. For example, a table with a Plate and Apple resting on top of it will cause both the Plate and Apple to fall to the floor if the table object supporting them is disabled.

❌ Remove

`RemoveFromScene` completely destroys an removes an object from the scene. This action cannot be undone (without resetting the scene).

```
controller.step(
    action="RemoveFromScene",
    objectId="Mug|+0.25|-0.27|+1.05"
)
```

Remove from Scene Parameters

objectId: str **required**

The unique identifier of the object in the scene.

🦇 Disable

`DisableObject` disables an object from being visible in the scene. Unlike the RemoveFromScene action, this does not permanently destroy an object in the scene, as it can be toggled back on later in the episode using EnableObject.

```
controller.step(
    action="DisableObject",
    objectId="DiningTable|+1.0|+1.0|+1.0"
)
```

Disable Objet Parameters

objectId: str **required**

The string id of the sim object to disable. Note that this may cause unintended interactions with other objects after an object is disabled. For example, a table with a Plate and Apple resting on top of it will cause both the Plate and Apple to fall to the floor if the table object supporting them is disabled.

😎 Enable

`EnableObject` activates an object if it has been previously disabled by DisableObject.

```
controller.step(
    action="EnableObject",
    objectId="DiningTable|+1.0|+1.0|+1.0"
)
```

Enable Objet Parameters

objectId: str **required**

The string id of the sim object to reactivate. Note that this may cause unintended interactions with other objects after an object is reactivated. Enabled objects will return to their original location, which may cause clipping or weird collision with other objects if another sim object or the agent has moved into the area where the enabled object is.

NEW Big News! We've announced 🏠 ProcTHOR →

🍅 Material Randomization
💡 Lighting Randomization
🎩 Initial Random Spawn
🎨 Color Randomization

🍅 Material Randomization

Randomizes all the materials in the scene. Each object may only randomize to sensible materials from a similar material type. For instance, an `Apple`'s materials won't change to an `Orange`'s materials, and a `Chair`'s material won't change to a `Plant`'s material. Empirically, the rooms generated from material randomization look incredibly realistic in most cases. Upon calling reset, materials change back to their original values.

```
controller.step(
    action="RandomizeMaterials",
    useTrainMaterials=None,
    useValMaterials=None,
    useTestMaterials=None,
    inRoomTypes=None
)
```

이 동영상을 시청하려면 Vimeo에 로그인하세요.

먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.

로그인

Test it live on the demo →

Randomize Materials Parameters

useTrainMaterials: Optional[bool] = None

Specifies if the randomization should include materials found in training rooms. Training rooms consist of all of the following:

- Kitchens: $\mathrm{FloorPlan}[1:20]$
- Living Rooms: $\mathrm{FloorPlan}[201:220]$
- Bedrooms: $\mathrm{FloorPlan}[301:320]$
- Bathrooms: $\mathrm{FloorPlan}[401:420]$
- RoboTHOR Train: $\mathrm{FloorPlan\_Train} * \_*$

If the agent is in one of the above scenes and `None` is passed in, then `useTrainMaterials` will default to `True`. Otherwise, it will default to `False`.

An exception is thrown if you are in a train scene, and you pass in `useTrainMaterials` as `False`. We do this because it guarantees every object can randomize to at least one material.

useValMaterials: Optional[bool] = None

Specifies if the randomization should include materials found in validation rooms. Validation rooms consist of all of the following:

- Kitchens: $\mathrm{FloorPlan}[21:25]$
- Living Rooms: $\mathrm{FloorPlan}[221:225]$
- Bedrooms: $\mathrm{FloorPlan}[321:325]$
- Bathrooms: $\mathrm{FloorPlan}[421:425]$
- RoboTHOR Val: $\mathrm{FloorPlan\_Val} * \_*$

If the agent is in one of the above scenes and `None` is passed in, then `useValMaterials` will default to `True`. Otherwise, it will default to `False`.

An exception is thrown if you are in a validation scene, and you pass in `useValMaterials` as `False`. We do this because it guarantees every object can randomize to at least one material.

useTestMaterials: Optional[bool] = None

Specifies if the randomization should include materials found in testing rooms. Testing rooms consist of all of the following:

- Kitchens: $\mathrm{FloorPlan}[26:30]$
- Living Rooms: $\mathrm{FloorPlan}[226:230]$
- Bedrooms: $\mathrm{FloorPlan}[326:330]$
- Bathrooms: $\mathrm{FloorPlan}[426:430]$

If the agent is in one of the above scenes and `None` is passed in, then `useTestMaterials` will default to `True`. Otherwise, it will default to `False`.

An exception is thrown if you are in a testing scene, and you pass in `useTestMaterials` as `False`. We do this because it guarantees every object can randomize to at least one material.

inRoomTypes: List[str] = None

Specifies which room types the randomizable materials may come from.

Valid room types include `Kitchen`, `LivingRoom`, `Bedroom`, `Bathroom`, and `RoboTHOR`. Casing is ignored. An exception is thrown if an unknown room type is passed in.

If `None` is passed in, then all room types will be included.

An exception is thrown if the current room type is not included in `inRoomTypes` and `None` is not passed in. We do this because it guarantees every object can randomize to at least one material.

💡 Lighting Randomization

```
controller.step(
    action="RandomizeLighting",
    brightness=(0.5, 1.5),
    randomizeColor=True,
    hue=(0, 1),
    saturation=(0.5, 1),
    synchronized=False
)
```

이 동영상을 시청하려면 Vimeo에 로그인하세요.

먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.

로그인

Test it live on the demo →

Randomize Lighting Parameters

brightness: Tuple[float, float] = (0.5, 1.5)

Sets the bounds with which a light's intensity may be multiplied by. Higher values are brighter. Both values must be greater than 0. The order of the values does not matter.

randomizeColor: bool = True

Specifies if the color should be randomized. If set to `False`, only each lights intensity may change.

hue: Tuple[float, float] = (0, 1)

Sets the $(\min, \max)$ domain of possible hue values for each light, which affects its color. Valid values are ranges between $[0:1]$, where:

- 0 maps to a hue of 0 degrees (i.e., red).
- 0.5 maps to a hue of 180 degrees (i.e., green).
- 1 maps to a hue of 360 degrees (i.e., red).

saturation: Tuple[float, float] = (0.5, 1)

Sets the $(\min, \max)$ domain of possible saturation values for each light. Valid values are ranges between $[0:1]$, where 0 corresponds to grayscale and 1 corresponds to full saturation.

synchronized: bool = False

Specifies if all lights in the room should be randomized with the same values (i.e., intensity and color). When `False`, each lighting object gets its own independent randomized intensity and randomized color.

🎩 Initial Random Spawn

`InitialRandomSpawn` attempts to randomize the position of `Pickupable` objects, placing them in any valid receptacle they could be placed in within the scene. Objects are not guaranteed to move from their default position, as sometimes there is not enough free space in the scene for an object to move.

> **Warning**
>
> After calling initial random spawn, each object will have its objectId recomputed (in the form of "objectType|x|y|z"). Thus, objects that move will have their objectId's changed. This is necessary to prevent collisions between objectId's, since initial random spawn may be used to duplicate existing objects in the scene.

```
controller.step(
    action="InitialRandomSpawn",
    randomSeed=0,
    forceVisible=False,
    numPlacementAttempts=5,
    placeStationary=True,
    numDuplicatesOfType = [
        {
            "objectType": "Statue"
             "count": 20
        },
        {
            "objectType": "Bowl"
            "count": 20,
        }
    ],
    excludedReceptacles=["CounterTop", "DiningTable"],
    excludedObjectIds=["Apple|1|1|2"]
)
```

Initial Random Spawn Parameters

randomSeed: int = 0

Used to seed the randomization for duplicatable scene states. Because this seed depends on the current state of the scene, remember to reset the scene with controller.reset() before running InitialRandomSpawn(), otherwise the seeded randomization will not be accurate.

forceVisible: bool = False

When enabled, the scene will attempt to randomize all moveable objects outside of receptacles in plain view. Use this if you want to avoid objects spawning out of view inside closed drawers, cabinets, etc.

numPlacementAttempts: int = 5

How many times each object in the scene attempts to randomly spawn. Setting this value higher will lead to fewer spawn failures at the cost of performance.

placeStationary: bool = True

Determines if spawned objects will be settled completely static and unmoving, or if non-determenistic physics resolve their final position. Setting this to False will allow physics to resolve final positions, which can be used to spawn an object on a sloped receptacle but have it end up rolling off.

numDuplicatesOfType: list[dict[str, any]] = []

Used to specify how many objects of a certain type will attempt to be duplicated somewhere in the scene. It does not guarantee this number of duplicated objects, only the number of attempted spawned objects, so this is the max it will be. This will only create copies of objects already in the scene, so if you request an object which is not in the scene upon reset, it will not work.

excludedReceptacles: list[str] = []

A list of receptacle object types to exclude from valid receptacles that can be randomly chosen as a spawn location. An example use case is excluding all CounterTop receptacles to allow for a scene configuration that has more free space on CounterTops in case you need free space for interaction. Note that this will not guarantee all listed receptacles as being completely clear of objects, as any objects that failed to reposition will remain in their default position, which might have been on the excluded receptacle type. Check the Actionable Properties section of the Objects documentation for a full list of Receptacle objects.

excludedObjectIds: list[str] = []

Used to prevent certain objects from changing in position.

> **Remark**
>
> This is often nice for rearrangement related tasks.

🎨 Color Randomization

```
controller.step(action="RandomizeColors")
```

이 동영상을 시청하려면 Vimeo에 로그인하세요.

먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.

로그인

Test it live on the demo →

Randomize Colors Description

Randomizes the color of each object in the scene. Compared to `RandomizeMaterials`, `RandomizeColors` looks much less realistic, but does enable a huge potential for visual variance.

Each color is uniformly sampled with a hue between $[0:1]$, a saturation in $[0.5:1]$, and a value in $[0.5:1]$.

🐘 Movement
🙃 Agent Rotation
🎥 Camera Rotation
🧘 Crouch
🧍 Stand
🎩 Teleport
😴 Done

🐘 Movement

Move the agent in a cardinal direction, relative to it's forward facing direction. Supported action directions are `MoveAhead`, `MoveBack`, `MoveLeft`, and `MoveRight`.

```python
controller.step(
    action="MoveAhead",
    moveMagnitude=None
)

# Other supported directions
controller.step("MoveBack")
controller.step("MoveLeft")
controller.step("MoveRight")
```

Movement Parameters

moveMagnitude: Optional[float] = None
Overrides the initialized step size of `gridSize`. The units are specified in meters. By default, the `gridSize` is initialized to 0.25 meters.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

🙃 Agent Rotation

Changes the yaw rotation of the agent's body. Supported action directions are `RotateRight` and `RotateLeft`.

```python
controller.step(
    action="RotateRight",
    degrees=None
)

# Other supported direction
controller.step("RotateLeft")
```

Rotate Parameters

degrees: Optional[float] = None
Overrides the initialized rotation amount of `rotateStepDegrees`.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

🎥 Camera Rotation

We can adjust the camera's pitch by utilizing the `LookUp` and `LookDown` commands. Both increment in 30° intervals, with angles being clamped between 60° in the downward direction and 30° in the upward direction.

```python
controller.step(
    action="LookUp",
    degrees=30
)

# Other supported direction
controller.step("LookDown")
```

Camera Rotation Parameters

degrees: float = 30
Specifies the number of degrees to look up or down. Must be greater than 0.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

🧘 Crouch

```python
controller.step(action="Crouch")
```

Crouch Description

Crouches the agent from a standing position. The standing position is roughly 0.22 meters taller than the crouching position.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

🧍 Stand

```python
controller.step(action="Stand")
```

Stand Description

Stands the agent from a crouching position. The crouching position is roughly 0.22 meters shorter than the standing position.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

🎩 Teleport

`Teleport` allows the agent to set its pose to anywhere in the scene in a single step. Valid poses do not place the agent outside the outer boundaries of the scene or in an area that it collides with an object.

```python
controller.step(
    action="Teleport",
    position=dict(x=1, y=0.9, z=-1.5),
    rotation=dict(x=0, y=270, z=0),
    horizon=30,
    standing=True
)
```

Teleport Parameters

position: Optional[dict[str, float]] = None
The position of the agent's body in global 3D coordinate space. If unspecified, the position of the agent remains the same.

rotation: Optional[dict[str, float]] = None
The rotation of the agent's body in global 3D space. Here, the rotation changes the agent's yaw rotation.

> **Remark**
> The default agent's body cannot change in pitch or roll; hence, why the $x, z = 0$.

If unspecified, the rotation of the agent remains the same.

horizon: Optional[float] = None
The `horizon` change's the camera's rotation. Values are clamped between [-30:60].

Since the agent looks up and down in 30° increments, it most common for the horizon to be in $\{-30, 0, 30, 60\}$.

> **Warning**
> Negative camera `horizon` values correspond to agent looking up, whereas positive `horizon` values correspond to the agent looking down.

If unspecified, the horizon of the agent remains the same.

standing: Optional[bool] = None
Denotes whether the agent is in a standing or crouched position. If unspecified, the standing state of the agent remains the same.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

In addition to being able to `Teleport`, we also have an equivalent action `TeleportFull` that strictly requires every degree of freedom to be specified. The strictness of this action is often useful in large projects, where we don't want to any part of the agent implied implicitly.

> **Warning**
> `TeleportFull` does not guarantee backwards compatibility in future releases. If a new degree of freedom is added to the agent, it will be added as a required parameter. If this is an issue, we recommend using `Teleport`.

```python
controller.step(
    action="TeleportFull",
    position=dict(x=1, y=0.9, z=-1.5),
    rotation=dict(x=0, y=270, z=0),
    horizon=30,
    standing=True
)
```

TeleportFull Parameters

position: dict[str, float] **required**
The position of the agent's body in global 3D coordinate space.

rotation: dict[str, float] **required**
The rotation of the agent's body in global 3D space. Here, the rotation changes the agent's yaw rotation.

> **Remark**
> The default agent's body cannot change in pitch or roll; hence, why the $x, z = 0$.

horizon: float **required**
The `horizon` change's the camera's rotation. Values are clamped between [-30:60].

Since the agent looks up and down in 30° increments, it most common for the horizon to be in $\{-30, 0, 30, 60\}$.

> **Warning**
> Negative camera `horizon` values correspond to agent looking up, whereas positive `horizon` values correspond to the agent looking down.

standing: bool **required**
Denotes whether the agent is in a standing or crouched position.

It is often useful to randomize the position of the agent in the scene, before starting an episode. Here, we can use:

1. GetReachablePositions, which does an optimized BFS over a grid spaced out by the initialized `gridSize`. The valid positions are then added and returned in a list.
2. `Teleport`, which can take a given position, and transform the agent to that position.

The process is illustrated below:

Step 1: Get the Positions

```python
positions = controller.step(
    action="GetReachablePositions"
).metadata["actionReturn"]
```

Response

```
[
    dict(x=(...), y=(...), z=(...)),
    dict(x=(...), y=(...), z=(...)),
    dict(x=(...), y=(...), z=(...)),
    {...}
    dict(x=(...), y=(...), z=(...)),
]
```

Step 2: Teleport to a Position

```python
import random
position = random.choice(positions)
controller.step(
    action="Teleport",
    position=position
)
```

Get Reachable Positions Response

actionReturn: list[dict[str, float]]
A list of each position that the agent can be at, without colliding into any other objects or going beyond the walls.

😴 Done

The `Done` action nothing to the state of the environment. But, it returns a cleaned up event with respect to the metadata.

```python
controller.step(action="Done")
```

It is often used in the definition of a successful navigation task (see Anderson et al.), where the agent must call the done action to signal that it knows that it's done, rather than arbitrarily or biasedly guessing.

> **Warning**
> The `Done` action does literally nothing to the state of the environment. For instance, if the agent calls `Done` and then `MoveAhead`, the `Done` action will have no affect on preventing a `MoveAhead` action from executing.

It is often used to return a cleaned up version of the metadata.

Held Objects
Pushing Objects
Teleporting Objects

Held Objects

Pickup Object

`PickupObject` allows the agent to pick up an interactable object specified by its objectId. Compatible objects have `pickupable` as `True` in their object metadata and return their current state with `isPickedUp`. Note that the agent's hand must be clear of obstruction or the action will fail. If the target object being in the agent's hand would cause it to clip into the environment, the action will also fail.

Picked up objects can also obstruct the Agent's view of the environment since the Agent's hand is always in camera view, so know that picking up larger objects will obstruct the field of vision.

> **Warning**
> Certain objects are `receptacles` that can themselves be picked up. If a moveable receptacle is picked up while other Sim Objects are inside of it, the contained objects will be picked up with the moveable receptacle. Here, a sequences like " `Place` the `Egg` on the `Plate`, then to `PickUp` the `Plate` " would `PickUp` the `Plate` with `Egg` on it.

```
controller.step(
    action="PickupObject",
    objectId="Apple|1|1|1",
    forceAction=False,
    manualInteract=False
)
```

Pickup Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
Set to `True` to allow an object to be picked up regardless of if it is within the `visibilityDistance` of the agent.

manualInteract: bool = False
By default, objects picked up by the agent teleport into the agent's hand at a default position in front of the agent camera. Set this to `True` in order to instead pick up an object at the object's location. This allows the agent to manipulate the object via object manipulation actions without the abstraction of the picked up object teleporting to the agent's hand.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Put Object

`PutObject` attempts to put an object the agent is holding onto or into the target receptacle. Valid target receptacle objects have `receptacle` as `True` in their object metadata.

```
controller.step(
    action="PutObject",
    objectId="Apple|1|1|1",
    forceAction=False,
    placeStationary=True
)
```

Put Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
Enable to ignore any Receptacle Restrictions when attempting to place objects. Normally objects will fail to be put on a receptacle if that receptacle is not valid for the object. This will also ignore interaction range restrictions of the agent. Note this does not guarantee an object will be placed in a receptacle, as some objects will not fit inside a receptacle regardless of the default object restrictions.

placeStationary: bool = True
If `False`, a placed object will use the physics engine to resolve the final position. This means placing an object on an uneven surface may cause inconsistent results due to the object rolling around or even falling off of the target receptacle. Note that because of variances in physics resolution, this placement mode is non-deterministic! If `True`, the object will be placed in/on the valid receptacle without using physics to resolve the final position. This means that the object will be placed so that it will not roll around. For deterministic placement make sure to set to `True`!

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Drop Object

`DropHandObject` attempts to drop an object currently in the agent's hand and let physics resolve where it lands. The action is different from `PutObject`, as this does not guarantee the held object will be put into a specified receptacle. This is meant to be used in tandem with the Move/Rotate Hand functions to maneuver a held object to a target area, and the let it drop.

Additionally, this drop action will fail if the held object is not clear from all collisions. Most importantly, the agent's collision will prevent drop, as dropping an object if it is "inside" the agent will lead to unintended behavior.

```
controller.step(
    action="DropHandObject",
    forceAction=False
)
```

Drop Object Parameters

forceAction: bool = False
Set to `True` to forcibly drop an object even if this would cause clipping with the environment, other objects, or the agent.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Throw Object

`ThrowObject` is an extension of `DropHandObject`. It throws the object currently in the agent's hand in the forward facing direction of the agent with a force of `moveMagnitude` newtons. Since objects have different mass properties, objects require different forces to throw them same distance.

```
controller.step(
    action="ThrowObject",
    moveMagnitude=150.0,
    forceAction=False
)
```

Throw Object Parameters

moveMagnitude: float — required
The amount of force used to throw the object in newtons. Note that objects of different masses will have different throw distances if this magnitude is not changed.

forceAction: bool = False
Set to `True` to forcibly throw an object even if this would cause clipping with the environment or other objects.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Move Held Object

While the agent is holding an object, it has several available actions to manipulate it. One such action is the ability to move the held object closer or further away from it. Held object movement is useful if we want to drop an object on a surface that is relatively far away.

There are several directions that the agent can move the held object. They include: `MoveHeldObjectAhead`, `MoveHeldObjectBack`, `MoveHeldObjectLeft`, `MoveHeldObjectRight`, `MoveHeldObjectUp`, and `MoveHeldObjectDown`. Each direction supports the same additional parameters.

> **Warning**
> After calling a move action (e.g., `MoveAhead`) or a rotate action (e.g., `RotateRight`) the position of the held object will reset to its default position in-front of the agent.

```
controller.step(
    action="MoveHeldObjectAhead",
    moveMagnitude=0.1,
    forceVisible=False
)

# Other supported directions
controller.step("MoveHeldObjectBack")
controller.step("MoveHeldObjectLeft")
controller.step("MoveHeldObjectRight")
controller.step("MoveHeldObjectUp")
controller.step("MoveHeldObjectDown")
```

Move Held Object Parameters

moveMagnitude: float — required
The distance, in meters, to move the held object in the specified direction.

forceVisible: bool = False
Setting `forceVisible=True` results in action failing if the object is not visible to the agent. This prevents the agent from hiding the held object inside or behind another object, or moving it too far, such that it's out of the agent's field of view.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

We also provide a separate helper action, `MoveHeldObject`, which allows the held object to move in several directions with only a single action:

```
controller.step(
    action="MoveHeldObject",
    ahead=0.1,
    right=0.05,
    up=0.12,
    forceVisible=False
)
```

Move Held Object Parameters

ahead: float — required
The distance, in meters, to move the held object forward, from the agent's current facing direction.

right: float — required
The distance, in meters, to move the held object rightwards, from the agent's current facing direction.

up: float — required
The distance, in meters, to move the held object upwards, from the agent's current facing direction.

forceVisible: bool = False
Setting `forceVisible=True` results in action failing if the object is not visible to the agent. This prevents the agent from hiding the held object inside or behind another object, or moving it too far, such that it's out of the agent's field of view.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Rotate Held Object

`RotateHeldObject` attempts to rotate the held object relative to its current rotation.

> **Warning**
> After calling a move action (e.g., `MoveAhead`) or a rotate action (e.g., `RotateRight`) the rotation of the held object will reset to its default rotation.

```
controller.step(
    action="RotateHeldObject",
    pitch=90,
    yaw=25,
    roll=45
)
```

Rotate Held Object Parameters

pitch: float = 0 — required
Increments the pitch of an object from the agent's current facing direction. Specified in degrees. The video below shows the held object's `pitch` incrementing in 15 degree increments.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

yaw: float = 0 — required
Increments the yaw of an object from the agent's current facing direction. Specified in degrees. The video below shows the held object's `yaw` incrementing in 15 degree increments.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

roll: float = 0 — required
Increments the roll of an object from the agent's current facing direction. Specified in degrees. The video below shows the held object's `roll` incrementing in 15 degree increments.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

An overload of `RotateHeldObject` can be used to rotate the held object to a fixed rotation.

```
controller.step(
    action="RotateHeldObject",
    rotation=dict(x=90, y=15, z=25)
)
```

Rotate Held Object Parameters

rotation: dict[str, float] — required
Sets the rotation of the object to the provided rotation, in degrees. The rotation is relative to the object's axes. So, the set rotation is independent of the object's current rotation. Rotations are specified in degrees.

Pushing Objects

Directional Push

`DirectionalPush` attempts to push an object in a given direction. Since objects can have different mass properties, pushing different objects often requires differing amounts of force to move them the same distance. Only `pickupable` and `moveable` objects can be pushed.

```
controller.step
    action="DirectionalPush",
    objectId="Sofa|3|2|1",
    moveMagnitude="100",
    pushAngle="90"
)
```

Directional Push Parameters

moveMagnitude: float — required
The amount of force used to pull the object in newtons. Following natural physics, objects of different masses may move different distances with the same amount of force.

pushAngle: float — required
The direction vector to push the object. Values in $[0 : 360]$ are valid, with 0 being the current forward direction of the agent. This value will change the push direction clockwise from the agent's forward. (i.e., 90 will push the object directly right, a value of 180 will push the object backwards).

objectId: str — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
Set to `True` to allow an object to be picked up regardless of if it is within the `visibilityDistance` of the agent.

Push Object

`PushObject` is syntactic sugar for `DirectionalPush` with a `pushAngle` of 0.

```
controller.step(
    action="PushObject",
    objectId="Mug|0.25|-0.27",
    forceAction=False
)
```

Push Object Parameters

moveMagnitude: float — required
The amount of force used to push the object in newtons. Note that objects of different masses will move different distances if this magnitude is not changed.

objectId: str — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
Set to `True` to allow an object to be picked up regardless of if it is within the `visibilityDistance` of the agent.

Pull Object

`PullObject` is syntactic sugar for `DirectionalPush` with a `pushAngle` of 180.

```
controller.step(
    action="PullObject",
    objectId="Mug|0.25|-0.27",
    forceAction=False
)
```

Push Object Parameters

moveMagnitude: float — required
The amount of force used to pull the object in newtons. Note that objects of different masses will move different distances if this magnitude is not changed.

objectId: str — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
Set to `True` to allow an object to be picked up regardless of if it is within the `visibilityDistance` of the agent.

Touch Then Apply Force

`TouchThenApplyForce` allows the agent to push an object at a certain point on the object, in a given direction. If a `pickupable` or `moveable` sim object is hit along the path of this ray, it will have a force determined by `moveMagnitude` applied to it instantaneously. This action will return feedback in the `actionReturn` attribute of the event metadata return.

```
event = controller.step(
    action="TouchThenApplyForce",
    x=0.5,
    y=0.5,
    direction={
        "x": 0.0,
        "y": 1.0,
        "z": 0.0
    },
    moveMagnitude=80,
    handDistance=1.5
)
```

Parameters

x: float — required
Used instead of `objectId` as an alternate way to target an object based on the last image frame. Valid values are in $[0 : 1]$, corresponding to how far left the image is from the previous frame.

y: float — required
Used in tandem with `x` to target an object based on the last image frame, rather than an `objectId`. Valid values are in $[0 : 1]$, corresponding to how from the top the object is from the previous frame.

direction: dict[str, float] — required
The direction vector relative to the agent's current forward to push any object touched.

moveMagnitude: float — required
The amount of force to apply to a touched object in newtons.

handDistance: float — required
The maximum Euclidean distance from the agent's camera that the (x, y) point that targets the object can be, in meters. If the point on the object is further away than `handDistance`, the action will fail.

Response

```
{
    "didHandTouchSomething": True
    "objectId": "Apple|+1|+1|+1"
    "armsLength": 1.20
}
```

event.metadata["actionReturn"]

didHandTouchSomething: bool
Returns `True` if a sim object was touched within the ray's length of `handDistance` meters.

objectId: str
The unique string id of the object touched by the ray. If the object touched in the scene has collision but is not a sim object, this `objectId` attribute will be "not a sim object, a structure was touched".

armsLength: float
The distance from the touched point on the object to the agent's camera. This distance will never exceed the handDistance value passed in originally unless the action finishes as a failed action (see below).

This action will return a failure (event.metadata["lastActionSuccess"] == False) only in two cases:

1. The raycast hit an object, but the object was outside of the maximum visibility range of the agent (ie: a handDistance of 10 is passed in, and hits an object 9 meters away, but the agent's max visibility distance is 1.5 meters, causing a failure). The feedback object generated in this case will be didHandTouchSomething=False, objectId="", armsLength=handDistance.
2. The handDistance of the action is larger than the agent's max visibility distance and no object was hit. The feedback object generated in this case will be didHandTouchSomething=False, objectId="", armsLength=handDistance with a metadata errormessage of "the position the hand would have moved to is outside the agent's max interaction range".

Note that this action interacts with the visibility of an object in order to determine what can be poked. The visibility of an object is defined by:

1. An object must be within the agent camera's field of view.
2. An object must be within the area of a cylinder defined by a radius of length visibilityDistance around the agent's vertical y axis.
3. If that object is within the cylinder, a line must be able to be cast from the agent's camera position to a point on that object unobstructed.

Because visibility is defined by the cylinder with radius visibilityDistance, the total area of objects that are touchable by this action is the intersection of the sphere of radius handDistance centered around the agent's camera, and the cylinder of radius visibilityDistance about the agent's vertical axis. Note that the agent camera is also centered around the agent's vertical axis.

Teleporting Objects

Place Object At Point

`PlaceObjectAtPoint` attempts to place an object flush with the surface of a receptacle. This can only be used on objects that are `pickupable` or `moveable`. The point to place an object at can be generated by the `GetSpawnCoordinatesAboveReceptacle` action below. Combine these two actions in order to teleport objects onto different receptacle surfaces.

```
controller.step(
    action="PlaceObjectAtPoint",
    objectId="Toaster|1|1|1",
    position={
        "x": -1.25,
        "y": 1.0,
        "z": -1.0
    }
)
```

Place Object At Point Parameters

objectId: str — required
The target object's `objectId`, found in the object's metadata.

position: dict[str, float] — required
Global (x, y, z) coordinates for the point to try and place the object.

Get Spawn Coordinate Above Receptacle

`GetSpawnCoordinatesAboveReceptacle` is to explicitly be used in tandem with the `PlaceObjectAtPoint` action. It returns an array of xyz dictionary representing the (x, y, z) coordinates of valid spawn positions above a `receptacle` object. The array of xyz dictionary is returned in the actionReturn metadata.

```
controller.step(
    action="GetSpawnCoordinatesAboveReceptacle",
    objectId="CounterTop|1|1|1",
    anywhere=False
)
```

Parameters

objectId: str — required
The target object's `objectId`, found in the object's metadata.

anywhere: bool = False
If `True`, spawn coordinates will be returned even if the exact position of the coordinate is outside of the agent's field of view. Keep False to return spawn coordinates even if they are not in view of the agent.

Open Object
Close Object
Break Object
Cook Object
Slice Object
Toggle Object
Dirty Object
Fill Object with Liquid
Empty Liquid from Object
Use Up Object

Open Object

`OpenObject` attempts to open an Openable object to a specified amount, parameterized by `openness`. The full list of openable object types can be filtered on the Object Types page.

Warning
An object can fail to open if it hits another object as it is opening. In this case the action will fail and the target object will reset to the position it was last in.

```
event = controller.step(
    action="OpenObject",
    objectId="Book|0.25|-0.27|0.95",
    openness=1,
    forceAction=False
)
```

Open Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

openness: float = 1
The proportion regarding how far the object should open. Valid values are in $[0:1]$, where the value 0 corresponds to completely closed, 1 corresponds to completely open, and 0.5 corresponds to halfway open, for instance.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to the state of its openness.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "openable": True,
    "openness": 0.75,
    "isOpen": True,

    {...}
}
```

Open Object Response

openable: bool
Can this object be opened?

openness: float
The proportion that the object is open, linearly scaled to the range $[0:1]$. For instance, if an object is a quarter of the way open, then its openness would be 0.25.

isOpen: bool
`False` when the object's openness is 0, `True` otherwise.

Close Object

`CloseObject` is syntactic sugar for `OpenObject` with an `openness` of 0.

Warning
An object can fail to close if it hits another object as it is opening. In this case the action will fail and the target object will reset to the position it was last in.

```
controller.step(
    action="CloseObject",
    objectId="Book|0.25|-0.27|0.95",
    forceAction=False
)
```

Close Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Break Object

`BreakObject` attempts to break a breakable object. If successful, the object appears visibly different. For instance, breakable objects may shatter completely into pieces or have their screens cracked. The full list of breakable object types can be filtered on the Object Types page. Broken objects cannot be unbroken (until the scene has reset).

```
event = controller.step(
    action="BreakObject",
    objectId="Vase|0.25|-0.27|0.95",
    forceAction=False
)
```

Break Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is broken.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "breakable": True,
    "isBroken": False,

    {...}
}
```

Break Object Response

breakable: bool
Can this object break?

isBroken: bool
Is this object currently broken?

Cook Object

`CookObject` attempts to switch an object to its cooked state. Objects cannot be uncooked (unless the scene is reset). The full list of cookable object types can be filtered on the Object Types page.

```
event = controller.step(
    action="CookObject",
    objectId="Egg|0.25|-0.27|0.95",
    forceAction=False
)
```

Cook Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is cooked.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "cookable": True,
    "isCooked": False,

    {...}
}
```

Cook Object Response

cookable: bool
Can this object be cooked?

isCooked: bool
Is this object currently cooked?

Slice Object

`SliceObject` attempts to slice an object.

If an object, such as an Apple, is successfully sliced, there may include several new AppleSliced objects in the scene and metadata, one for each slice. Other objects, like Egg, may only break after being sliced. Sliced objects cannot be unsliced (unless the scene is reset).

The full list of sliceable objects can be filtered on the Object Types page.

```
event = controller.step(
    action="SliceObject",
    objectId="Potato|0.25|-0.27|0.95",
    forceAction=False
)
```

Slice Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is sliced.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "sliceable": True,
    "isSliced": False,

    {...}
}
```

Slice Object Response

sliceable: bool
Can this object be sliced?

isSliced: bool
Is this object currently sliced?

Toggle Object

`ToggleObjectOn` and `ToggleObjectOff` toggles an object between on and off states, respectively. Examples include Lamps, Light Switches, Stove Knobs, and Laptops. The full list of toggleable objects can be filtered on the Object Types page.

Toggle On

```
event = controller.step(
    action="ToggleObjectOn",
    objectId="LightSwitch|0.25|-0.27|0.95",
    forceAction=False
)
```

Toggle Off

```
event = controller.step(
    action="ToggleObjectOff",
    objectId="LightSwitch|0.25|-0.27|0.95",
    forceAction=False
)
```

Toggle Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is toggled.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "toggleable": True,
    "isToggled": True,

    {...}
}
```

Toggle Object Response

toggleable: bool
Can this object be toggled?

isToggled: bool
Is this object currently toggled on?

Dirty Object

`DirtyObject` attempts to make an object look dirty. `CleanObject` can then be used to make the object look clean again. The full list of dirtyable object types can be filtered on the Object Types page.

Dirty Object

```
event = controller.step(
    action="DirtyObject",
    objectId="Mug|0.25|-0.27|0.95",
    forceAction=False
)
```

Clean Object

```
event = controller.step(
    action="CleanObject",
    objectId="Mug|0.25|-0.27|0.95",
    forceAction=False
)
```

Dirty Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is toggled.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "toggleable": True,
    "isToggled": True,

    {...}
}
```

Dirty Object Response

toggleable: bool
Can this object be toggled?

isToggled: bool
Is this object currently toggled on?

Fill Object with Liquid

`FillObjectWithLiquid` attempts to fill an object with `fillLiquid`. Only compatible objects that are empty can be filled. The full list of object types that can be filled with liquid can be filtered on the Object Types page.

```
event = controller.step(
    action="FillObjectWithLiquid",
    objectId="Mug|0.25|-0.27|0.95",
    fillLiquid="coffee",
    forceAction=False
)
```

Fill Object with Liquid Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

fillLiquid: string — required
The type of liquid that fills the object. Valid liquids are `"coffee"`, `"wine"`, and `"water"`.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is filled with liquid.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "fillLiquid": "coffee",
    "canFillWithLiquid": True,
    "isFilledWithLiquid": True,

    {...}
}
```

Fill Object with Liquid Response

fillLiquid: Optional[str]
Which liquid is the object filled with? If the object is not filled with liquid, it will report `None`. Valid liquids are `"coffee"`, `"wine"`, and `"water"`.

canFillWithLiquid: bool
Can this object be filled with liquid?

isFilledWithLiquid: bool
Is this object currently filled with liquid?

Empty Liquid from Object

`EmptyLiquidFromObject` attempts to empty the liquid from an object if it is currently filled with liquid (i.e., `isFilledWithLiquid` is True).

Remark
If an object is rotated downward far enough, gravity may cause the object to lose its liquid automatically, causing `isFilledWithLiquid` to be `False`. Here, the liquid will not appear spilled on the ground like a puddle. Instead, it disappears.

```
event = controller.step(
    action="EmptyLiquidFromObject",
    objectId="Mug|0.25|-0.27|0.95",
    forceAction=False
)
```

Empty Liquid From Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it can be emptied of liquid.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "fillLiquid": None,
    "canFillWithLiquid": True,
    "isFilledWithLiquid": False,

    {...}
}
```

Empty Liquid from Object Response

fillLiquid: Optional[str]
Which liquid is the object filled with? If the object is not filled with liquid, it will report `None`. Valid liquids are `"coffee"`, `"wine"`, and `"water"`.

canFillWithLiquid: bool
Can this object be emptied of liquid?

isFilledWithLiquid: bool
Is this object currently filled with liquid?

Use Up Object

`UseUpObject` attempts to use up parts of an object. The action works with objects like ToiletPaper, TissueBox, and PaperTowelRoll, which each has an full/empty state. The full list of object types that can be used up can be filtered on the Object Types page.

```
event = controller.step(
    action="UseUpObject",
    objectId="ToiletPaper|0.25|-0.27|0.95",
    forceAction=False
)
```

Use Up Object Parameters

objectId: string — required
The target object's `objectId`, found in the object's metadata.

forceAction: bool = False
If `False`, the agent will not be able to interact with the object unless it is within the initialized `visibilityDistance` of the object and the object appears in the agent's current frame. This prevents the agent from unnaturally interacting with objects that are too far away.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

Each object contains metadata in the event pertaining to if it is used up.

```
event.metadata["objects"][i]
```

Example Response

```
{
    "canBeUsedUp": True,
    "isUsedUp": False,

    {...}
}
```

Use Up Object Response

canBeUsedUp: bool
Can this object be used up?

isUsedUp: bool
Is this object currently used up?